UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

VALDINEI BUCMAIER SANTOS

MIGRAÇÃO DE UMA *ENGINE* DE XADREZ EM JAVA PARA C# COM ESTUDO COMPARATIVO DO DESEMPENHO DE EXECUÇÃO DO ALGORITMO MINIMAX COM PODAS ALFA-BETA

TRABALHO DE CONCLUSÃO DE CURSO

PATO BRANCO
2020

VALDINEI BUCMAIER SANTOS

MIGRAÇÃO DE UMA *ENGINE* DE XADREZ EM JAVA PARA C# COM ESTUDO COMPARATIVO DO DESEMPENHO DE EXECUÇÃO DO ALGORITMO MINIMAX COM PODAS ALFA-BETA

Trabalho de Conclusão de Curso de graduação, apresentado à disciplina de Trabalho de Conclusão de Curso 2, do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas, da Universidade Tecnológica Federal do Paraná, Câmpus Pato Branco, como requisito parcial para obtenção do título de Tecnólogo.

Orientador: Dr. Luis Carlos Ferreira Bueno

PATO BRANCO
2020

Ministério da Educação
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
UTFPR - CAMPUS PATO BRANCO
DIRETORIA GERAL - CÂMPUS PATO BRANCO
DIR. DE GRAD. E EDUCAÇÃO PROFISSIONAL-PB
DEP. ACADÊMICO DE INFORMÁTICA DAINF-PB

TERMO DE APROVAÇÃO

TRABALHO DE CONCLUSÃO DE CURSO

Migração de uma Engine de Xadrez em Java Para C# com Estudo Comparativo do Desempenho de Execução do Algoritmo Minimax com Podas Alfa-Beta

POR

Valdinei Bucmaier Santos

Este trabalho de conclusão de curso foi apresentado no dia 24 de novembro de 2020, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas, pela Universidade Tecnológica Federal do Paraná. O acadêmico foi arguido pela Banca Examinadora composta pelos professores abaixo assinados. Após deliberação, a Banca Examinadora considerou o trabalho aprovado.

Banca examinadora:

Prof. Dr. Luis Carlos Ferreira Bueno	Profa. Dra. Kathya Silvia Collazos Linares	Prof. MSc Vinicius Pegorini
Professor orientador	Professora convidada	Professor convidado

Assinam também:

Prof. Dr. Edilson Pontarolo	Profa. Dra. Mariza Miola Dosciatti
Coordenador do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas	Responsável pela Atividade de Trabalho de Conclusão de Curso

Documento assinado eletronicamente por (Document electronically signed by) **LUIS CARLOS FERREIRA BUENO, PROFESSOR DO MAGISTERIO SUPERIOR**, em (at) 03/12/2020, às 20:01, conforme horário oficial de Brasília (according to official Brasilia-Brazil time), com fundamento no (with legal based on) art. 6º, § 1º, do Decreto nº 8.539, de 8 de outubro de 2015.

Documento assinado eletronicamente por (Document electronically signed by) **VINICIUS PEGORINI, PROFESSOR ENS BASICO TECN TECNOLOGICO**, em (at) 04/12/2020, às 08:21, conforme horário oficial de Brasília (according to official Brasilia-Brazil time), com fundamento no (with legal based on) art. 6º, § 1º, do Decreto nº 8.539, de 8 de outubro de 2015.

Documento assinado eletronicamente por (Document electronically signed by) **MARIZA MIOLA DOSCIATTI, PROFESSOR ENS BASICO TECN TECNOLOGICO**, em (at) 04/12/2020, às 09:34, conforme horário oficial de Brasília (according to official Brasilia-Brazil time), com fundamento no (with legal based on) art. 6º, § 1º, do Decreto nº 8.539, de 8 de outubro de 2015.

Documento assinado eletronicamente por (Document electronically signed by) **KATHYA SILVIA COLLAZOS LINARES, PROFESSOR DO MAGISTERIO SUPERIOR**, em (at) 04/12/2020, às 11:56, conforme horário oficial de Brasília (according to official Brasilia-Brazil time), com fundamento no (with legal based on) art. 6º, § 1º, do Decreto nº 8.539, de 8 de outubro de 2015.

Documento assinado eletronicamente por (Document electronically signed by) **EDILSON PONTAROLO, COORDENADOR(A) DE CURSO/PROGRAMA**, em (at) 07/12/2020, às 10:28, conforme horário oficial de Brasília (according to official Brasilia-Brazil time), com fundamento no (with legal based on) art. 6º, § 1º, do Decreto nº 8.539, de 8 de outubro de 2015.

A autenticidade deste documento pode ser conferida no site (The authenticity of this document can be checked on the website) https://sei.utfpr.edu.br/sei/controlador_externo.php?acao=documento_conferir&id_orgao_acesso_externo=0, informando o código verificador (informing the verification code) **1771325** e o código CRC (and the CRC code) **E9CBB1F5**.

RESUMO

Este trabalho apresenta um estudo comparativo de desempenho de uma *engine* de xadrez em Java e outra em C#. A *engine* em C# foi construída a partir da migração de código em Java. Serve como guia para desenvolvedores que desejarem migrar aplicações e resultados numéricos de desempenho que podem ajudar na decisão de custo-benefício em casos de migração. Apresenta também as principais técnicas da IA que são utilizadas na construção de uma *engine* de jogos como o xadrez. A migração foi bem-sucedida, pois os movimentos resultantes em cada caso analisado em ambas as *engines* são idênticos. Testes de desempenho foram realizados considerando todas as fases do jogo (abertura, meio e final), em profundidades de busca 3 e 5. A *engine* Java se mostrou bastante superior em termos de desempenho.

Palavras-chave: Inteligência artificial. Xadrez. *Engine*. MINIMAX. C#.

ABSTRACT

This paper presents a comparative study of the performance of a chess engine in Java and another in C#. The engine in C# was built from code migration in Java. It serves as a guide for developers who want to migrate applications and numerical performance results that can help them make a cost-effective decision when migrating. It also presents the main AI techniques that are used in building a game engine like chess. The migration was successful, as the resulting movements in each case analyzed in both engines are identical. Performance tests were performed considering all phases of the game (opening, middle and final), in search depths 3 and 5. The Java engine proved to be quite superior in terms of performance.

Keywords: Artificial intelligence. Chess. Engine. MINIMAX. C#.

LISTA DE FIGURAS

Figura 1 - Algoritmo geral de busca em árvore e busca em grafo 19
Figura 2 - Pseudocódigo busca em largura em árvore binária simples 20
Figura 3 - Busca em largura em uma árvore binária simples 20
Figura 4 - Busca em profundidade em uma árvore binária 21
Figura 5 - Algoritmo de busca de aprofundamento iterativo 22
Figura 6 - Árvore de busca E-OU para mundo de aspirador de pó 23
Figura 7 - Árvore de busca (parcial) para jogo da velha 27
Figura 8 - Árvore de jogo de duas jogadas 28
Figura 9 - Algoritmo para calcular decisões MINIMAX 29
Figura 10 - Caso geral de poda alfa-beta 30
Figura 11 - Algoritmo de busca alfa-beta 31
Figura 12 - Diagrama de classes parte 1 39
Figura 13 - Diagrama de classes parte 2 40
Figura 14 - Tela principal do *software Arena Chess GUI* 41
Figura 15 - *Engine* Orion executando em sua tela de console 42
Figura 16 - Janela *Engine Management* 46
Figura 17 - Janela *Replace/add Engine* 47
Figura 18 - Tabuleiro com posição de início de jogo 49
Figura 19 - Tabuleiro com posição de meio de jogo 50
Figura 20 - Tabuleiro com posição de final de jogo 51
Figura 21 - Resultado primeiro movimento, profundidades 3 e 5 52
Figura 22 - Resultado abertura, profundidade 3 53
Figura 23 - Resultado abertura, profundidade 5 54
Figura 24 - Resultado meio de jogo, profundidade 3 55
Figura 25 - Resultado meio de jogo, profundidade 5 56
Figura 26 - Resultado final de jogo, profundidade 3 e 5 57

LISTA DE QUADROS

Quadro 1 - Métodos da classe BoardGeneration 37
Quadro 2 - Métodos da classe Moves 37
Quadro 3 - Métodos da classe PrincipalVariation 38
Quadro 4 - Código em Java e código correspondente em C# 44

LISTAGENS

Listagem 1 - Constantes da classe Bench 33
Listagem 2 - Exemplo de declaração dos vetores 34
Listagem 3 - Array de Stopwatch 34
Listagem 4 - Arrays para armazenar e localizar menor e maior tempo 35
Listagem 5 - de primeiro movimento, profundidade de busca 3 58
Listagem 6 - Teste de primeiro movimento, profundidade de busca 5 58
Listagem 7 - Teste de abertura, profundidade de busca 3 58
Listagem 8 - Teste de abertura, profundidade de busca 5 59
Listagem 9 - Teste de meio jogo, profundidade de busca 3 59
Listagem 10 - Teste de meio jogo, profundidade de busca 5 60
Listagem 11 - Teste de final de jogo, profundidade de busca 3 e 5 61
Listagem 12 - Erro código Java ao gravar no arquivo .txt 62

LISTA DE GRÁFICOS

Gráfico 1 - *Engine* Java: primeiro movimento, função evaluate, profundidade 3 63
Gráfico 2 - *Engine* C#: primeiro movimento, função evaluate, profundidade 3 63
Gráfico 3 - *Engine* Java: primeiro movimento, função zWSearch, profundidade 3 64
Gráfico 4 - *Engine* C#: primeiro movimento, função zWSearch, profundidade 3 64
Gráfico 5 - *Engine* Java: primeiro movimento, função evaluate, profundidade 5 65
Gráfico 6 - *Engine* C#: primeiro movimento, função evaluate, profundidade 5 65
Gráfico 7 - *Engine* Java: primeiro movimento, função zWSearch, profundidade 5 66
Gráfico 8 - *Engine* C#: primeiro movimento, função zWSearch, profundidade 5 66
Gráfico 9 - *Engine* Java: fase de abertura, função evaluate, profundidade 3 67
Gráfico 10 - *Engine* C#: fase de abertura, função evaluate, profundidade 3 67
Gráfico 11 - *Engine* Java: fase de abertura, função zWSearch, profundidade 3 68
Gráfico 12 - *Engine* C#: fase de abertura, função zWSearch, profundidade 3 68
Gráfico 13 - *Engine* Java: fase de abertura, função evaluate, profundidade 5 69
Gráfico 14 - *Engine* C#: fase de abertura, função evaluate, profundidade 5 69
Gráfico 15 - *Engine* Java: fase de abertura, função zWSearch, profundidade 5 70
Gráfico 16 - *Engine* C#: fase de abertura, função zWSearch, profundidade 5 70
Gráfico 17 - *Engine* Java: fase de meio jogo, função evaluate, profundidade 3 71
Gráfico 18 - *Engine* C#: fase de meio jogo, função evaluate, profundidade 3 71
Gráfico 19 - *Engine* Java: fase de meio jogo, função zWSearch, profundidade 3 72
Gráfico 20 - *Engine* C#: fase de meio jogo, função zWSearch, profundidade 3 72
Gráfico 21 - *Engine* Java: fase de meio jogo, função evaluate, profundidade 5 73
Gráfico 22 - *Engine* C#: fase de meio jogo, função evaluate, profundidade 5 73
Gráfico 23 - *Engine* Java: fase de meio jogo, função zWSearch, profundidade 5 74
Gráfico 24 - *Engine* C#: fase de meio jogo, função zWSearch, profundidade 5 74
Gráfico 25 - *Engine* Java: fase de final de jogo, função evaluate, profundidade 3 75
Gráfico 26 - *Engine* C#: fase de final jogo, função evaluate, profundidade 3 75
Gráfico 27 - *Engine* Java: fase de final de jogo, função zWSearch, profundidade 3 76
Gráfico 28 - *Engine* C#: fase de final de jogo, função zWSearch, profundidade 3 76
Gráfico 29 - *Engine* Java: fase de final de jogo, função evaluate, profundidade 5 77
Gráfico 30 - *Engine* C#: fase de final de jogo, função evaluate, profundidade 5 77
Gráfico 31 - *Engine* Java: fase de final de jogo, função zWSearch, profundidade 5 78

Gráfico 32 - *Engine* C#: fase de final de jogo, função zWSearch, profundidade 5 78

LISTA DE TABELAS

Tabela 1 – Diferenças de tempo e percentual de desempenho da *engine* C# com relação à Java 79
Tabela 2 - Número médio de posições avaliadas por segundo pelas engines 81
Tabela 3 - Resultados dos testes de desempenho: primeiro movimento com profundidade de busca 3 86
Tabela 4 - Resultados dos testes de desempenho: primeiro movimento com profundidade de busca 5 86
Tabela 5 - Resultados dos testes de desempenho: fase de abertura com profundidade de busca 3 87
Tabela 6 - Resultados dos testes de desempenho: fase de abertura com profundidade de busca 5 87
Tabela 7 - Resultados dos testes de desempenho: fase de meio jogo com profundidade de busca 3 88
Tabela 8 - Resultados dos testes de desempenho: fase de meio jogo com profundidade de busca 5 88
Tabela 9 - Resultados dos testes de desempenho: fase de final de jogo com profundidade de busca 3 89
Tabela 10 - Resultados dos testes de desempenho: fase de final de jogo com profundidade de busca 5 89

SUMÁRIO

1 INTRODUÇÃO 13
1.1 CONSIDERAÇÕES INICIAIS 13
1.2 OBJETIVOS 14
1.2.1 Objetivo Geral 14
1.2.2 Objetivos Específicos 15
1.3 JUSTIFICATIVA 15
1.4 ESTRUTURA DO TRABALHO 15
2 REFERENCIAL TEÓRICO 17
2.1 DEFINIÇÕES DE INTELIGÊNCIA ARTIFICIAL E SEUS FUNDAMENTOS 17
2.2 RESOLUÇÕES DE PROBLEMAS COM ALGORITMOS DE BUSCA 18
2.2.1 Estratégias de Busca sem Informação 19
2.2.1.1 Busca em largura 19
2.2.1.2 Busca em profundidade e com profundidade Limitada 20
2.2.1.3 Busca de aprofundamento iterativo 21
2.2.2 Estratégias de Busca com Informação 22
2.2.2.1 Busca gulosa de melhor escolha 22
2.2.2.2 Busca com ações não determinísticas 22
2.2.2.3 Árvores de busca E-OU 23
2.3 FUNCIONAMENTOS DAS *ENGINES* DE XADREZ 24
2.4 FORMALIZAÇÕES DAS HEURÍSTICAS NO XADREZ 25
2.5 BUSCA COMPETITIVA 26
2.5.1 Decisões Ótimas Em Jogos 27
2.5.2 O Algoritmo MINIMAX 28
2.5.3 Poda Alfa-Beta 29
3 MATERIAIS E MÉTODO 32
3.1 MATERIAIS 32
3.2 MÉTODO 32
4 RESULTADOS 36
4.1 ESCOPO DO SISTEMA 36
4.2 MODELAGEM DO SISTEMA 36
4.3 APRESENTAÇÃO DO SISTEMA 41

4.4 IMPLEMENTAÇÃO DO SISTEMA 43
4.4.1 Tradução do Código 43
4.4.2 Integração com *Arena* 45
4.4.3 Testes de Validação 51
4.4.4 Testes de Desempenho 57
5 DISCUSSÕES 79
6 CONCLUSÃO 82
6.1 LIMITAÇÕES 82
6.2 TRABALHOS FUTUROS 83
REFERÊNCIAS 84
APÊNDICE A 86
APÊNDICE B 91

1 INTRODUÇÃO

Este capítulo apresenta as considerações iniciais, objetivos, justificativa, e estrutura, relacionados ao trabalho de migração de uma *engine* de xadrez programada em Java para a linguagem C#, utilizando o algoritmo MINIMAX com poda alfa-beta.

1.1 CONSIDERAÇÕES INICIAIS

No final da década de noventa os noticiários pelo mundo apresentavam em suas manchetes que uma máquina havia alcançado a inteligência de um humano, e não a de um humano qualquer. No ano de 1997 o computador Deep-Blue vencia sua primeira partida de xadrez contra o campeão mundial, Gary Kasparov. (RUIZ; ALDERGUER; ROSIQUE; QUEVEDO; ANGEL, 1999, p. 101).

Conforme menciona Ruiz, Alderguer e Rosique (1999, p. 1) a humanidade tentou, durante séculos, construir e encontrar nas máquinas formas de facilitar tarefas complexas, demoradas, repetitivas ou perigosas, possibilitando, por exemplo, a maior precisão em cálculos. A ciência da computação veio avançando a ponto de projetar máquinas que imitavam o comportamento de animais e também as capazes de jogar muitos jogos, inclusive os de tática e fortemente baseado em regras, como é característica nos principais jogos de tabuleiro.

A parte da ciência da computação responsável pela construção de máquinas capazes de realizar tarefas humanas que exigem inteligência chama-se Inteligência Artificial (IA). (MINSKY, 1986 apud RUIZ; ALDERGUER; ROSIQUE; QUEVEDO; ANGEL, 1999, p. 2) [1]. A IA possui papel fundamental na área de desenvolvimento de jogos, tais como o jogo de tática por excelência: o xadrez.

Para alcançar a solução da melhor jogada em um determinado jogo é necessária a aplicação de técnicas e métodos de busca, o principal método a ser utilizado no código da *engine* do jogo de xadrez objeto desse projeto, será o método de busca MINIMAX, utilizando a técnica de poda alfa beta.

Conforme apresentado nos três parágrafos a seguir, Ruiz *et al.*, (1999, p. 102-104) trazem de forma resumida características dos algoritmos de busca que são utilizados em jogos e que estão presentes na *engine.*

[1] MINSKY, M. **Semantic Information Processing**. MIT Press, 1968.

O jogo deve ter a participação de dois jogadores, cada um tentando a vitória, podendo existir empate. Um jogador é caracterizado como sendo o MAX e o outro o MIN. O MAX sempre começa a jogar, e cada estado é uma configuração das peças posicionadas no tabuleiro. A decisão para realização da melhor jogada é calculada com base numa busca em um grafo, representado por uma árvore, onde cada nó representa uma posição de jogo (situação de tabuleiro), e cada aresta indica um movimento possível. Cada nó possui um fator de ramificação, ou seja, o número de movimentos válidos possíveis para a posição representada naquele nó. O primeiro nível de desdobramento do grafo é o MAX, o segundo o MIN, o terceiro MAX e segue-se de forma alternada até o número de níveis desejados.

A árvore de busca será gerada até determinado nível de profundidade e nesse nível deve-se aplicar uma função de avaliação nos nós finais da árvore. O valor mais alto retornado pela função será usado para escolher a melhor jogada a ser efetivada por MAX, maximizando sua chance de ganhar enquanto na rodada seguinte no nível MIN o outro jogador estará tentando minimizar a chance da vitória de MAX.

Aplicando a técnica de poda alfa beta ao tentar encontrar o valor MINIMAX determinam-se valor de limite de corte para um nó MIN, o alfa (limite inferior sendo igual ao mais alto valor atual de seus nós antecessores MAX) e para um nó MAX, o beta (limite superior sendo igual ao menor valor atual de seus nós antecessores MIN), não avaliando os nós ou ramificação que não são capazes de alterar esse valor. Isto diminui significativamente o número de nós que precisam ser avaliados. (SAKS, 1986).

Assim, duas tarefas serão executadas neste trabalho:

1. Migração de uma *engine* de xadrez em Java para C# usando o algoritmo MINIMAX com podas alfa-beta, a partir do modelo desenvolvido em Java por Jonathan Warkentin, *Advanced Chess Engine*, em seu canal no *YouTube*: *Logic Crazy Chess*.
2. Integração com uma interface gráfica restrita ao ambiente *desktop* para execução e visualização do jogo.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Realizar a migração de uma *engine* de xadrez do Java para C# utilizando algoritmo MINIMAX com poda alfa-beta, a fim de analisar o desempenho de ambas as linguagens em operações de busca na árvore do jogo.

1.2.2 Objetivos Específicos

- Migrar uma *engine* de xadrez construída em Java para C# usando algoritmo MINIMAX com poda alfa-beta.
- Integrar a *engine* com uma interface visando o correto funcionamento do jogo.
- Permitir ao jogador humano disputar partidas de xadrez contra a aplicação.
- Avaliar o desempenho das *engines* em termos de posições avaliadas por segundo.

1.3 JUSTIFICATIVA

O jogo de xadrez tem sido muito utilizado para o desenvolvimento da IA. Entretanto, para adquirir a compreensão de como os algoritmos de IA funcionam é importante o desenvolvimento de tais algoritmos na prática. Além disto, a prática e estudo do xadrez em si influem positivamente no desenvolvimento educacional e pessoal. (D'LUCIA; LEITÃO; FONSECA; SILVA; SCALVI, 2010, p. 98).

Existem poucas *engines* desenvolvidas em C#, portanto é interessante acrescentarmos este desenvolvimento ao pequeno grupo já existente, disponibilizando de forma gratuita o código, para que entusiastas e aprendizes da IA aprimorem seus conhecimentos. Também nenhum comparativo de desempenho entre *engines* desenvolvidas em Java e desenvolvidas em C# foi encontrado na literatura. Assim, este trabalho se propõe a realizar tal estudo comparativo.

1.4 ESTRUTURA DO TRABALHO

Neste capítulo foram apresentados os aspectos introdutórios, objetivos e justificativas do trabalho. O capítulo 2 apresenta uma revisão de literatura sobre o tema pertinente ao trabalho, bem como alguns trabalhos do uso dos conceitos de IA utilizados no xadrez na comunidade educacional. O capítulo 3 apresenta os materiais e a metodologia, para alcançar os objetivos propostos. No capítulo 4 encontram-se a modelagem do sistema, os resultados comparativos em nível de codificação entre Java e C# e o comparativo de desempenho entre

as *engines*. O capítulo 5 apresenta as discussões finais, vantagens e limitações da abordagem e os trabalhos futuros.

2 REFERENCIAL TEÓRICO

2.1 DEFINIÇÕES DE INTELIGÊNCIA ARTIFICIAL E SEUS FUNDAMENTOS

Conforme expõem Russell e Norvig (2013, p. 6) a Inteligência Artificial, é um campo de estudo em ciências e engenharia que visa algo além da compreensão de entidades inteligentes, pois estuda também formas de construir. Ela encontra-se presente em tarefas como as de tradução automática, reconhecimento de voz, combate a *spam*, jogos de xadrez assim como nas de diagnóstico de doenças e direção de veículos autônomos.

Historicamente as definições de IA dividem-se em quatro abordagens conforme Russell e Norvig (2013, p. 25) apresentam centradas em:

- Pensar como seres humanos: "O novo e interessante esforço para fazer os computadores pensarem (...) máquinas com mentes, no sentido total e literal" (HAUGELAND, 1985 apud RUSSELL; NORVIG, 2013, p. 25) [2].
- Agir como seres humanos: "O estudo de como os computadores podem fazer tarefas que hoje são melhor desempenhadas pelas pessoas" (RICH; KNIGHT, 1991 *apud* RUSSELL; NORVIG, 2013, p. 25)[3].
- Pensar racionalmente: "O estudo das faculdades mentais pelo uso de modelos computacionais" (CHARNIAK; MCDERMOTT, 1985 apud RUSSELL; NORVIG, 2013, p. 25)[4].
- Agir racionalmente: "Inteligência Computacional é o estudo do projeto de agentes inteligentes" (POOLE *et al*., 1998 *apud* RUSSELL; NORVIG, 2013, p. 25)[5].

Este projeto centra-se nesta última abordagem, onde o agente computacional age com intenção de chegar ao melhor resultado, ou havendo incerteza, busca alcançar o melhor resultado por ele esperado. O agente que joga xadrez atua em um ambiente multiagente, dois jogadores, e competitivo, onde um agente tenta maximizar o seu desempenho e minimizar o do adversário, baseado em objetivos, o principal é o estado de xeque-mate ao rei adversário. As ações realizadas são sequenciais, a ação atual afetará o estado do jogo e consequentemente as decisões futuras.

---

[2] HAUGELAND, John. **Artificial Intelligence: The Very Idea**. MIT Press, 1985.
[3] RICH, E; KNIGHT, K. **Artificial Intelligence**. 2.ed. McGraw-Hill, 1991.
[4] CHARNIAK, E; MCDERMOTT, D. **Introduction to Artificial Intelligence**. Addison-Wesley, 1985.
[5] POOLE, D. *et al.* **Computational intelligence: A logical approach**. Oxford University Press, 1998.

Várias disciplinas possuem papel importante para IA, dentre as citadas por Russell e Norvig (2013, p. 28-40). Entre elas, não se deve deixar de destacar as contribuições de filosofia, matemática, economia e engenharia de computação.

Os filósofos, dentre eles Descartes, com o que hoje é chamado de racionalismo, assemelharam a mente a uma máquina, no sentido de que internamente ela codifica o conhecimento e o utiliza para realizar as escolhas de suas ações. (RUSSEL; NORVIG, 2013 p. 56). Matemáticos foram os responsáveis por formalizar a lógica, computação e probabilidade, permitindo a criação e desenvolvimento dos algoritmos. As ciências econômicas com teorias como as da utilidade, da decisão e dos jogos apresentaram o problema de decidir maximizar os resultados em cada decisão, princípio de que os agentes são racionais. E, por fim, a engenharia da computação tem contribuído com o fornecimento de máquinas que permitem desenvolver as aplicações.

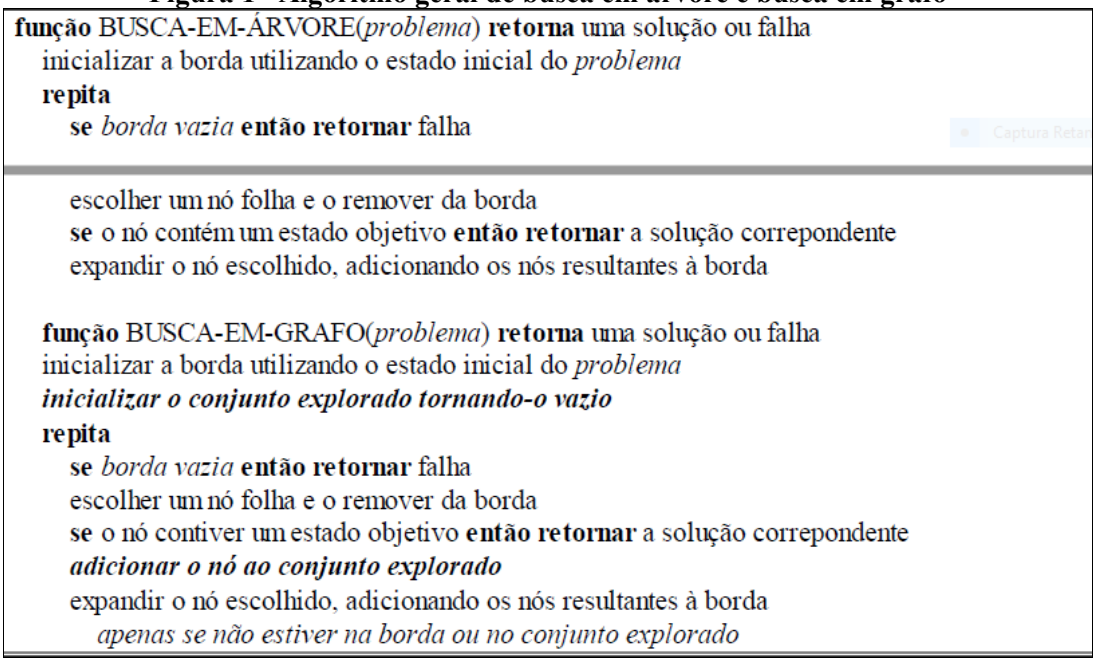
Portanto reforça-se que a Inteligência Artificial é um campo que atua e desenvolve-se constantemente em parceria com outras disciplinas.

2.2 RESOLUÇÕES DE PROBLEMAS COM ALGORITMOS DE BUSCA

Conforme definem Russell e Norvig (2013), o processo de procurar por uma sequência de ações que possa alcançar seu objetivo é chamado de busca. Os algoritmos de busca recebem o problema como entrada e devolvem uma solução sob a forma de uma sequência de ações, que depois de encontrada pode ser executada. Forma-se uma árvore de busca onde a raiz é o estado inicial, os ramos são as ações e os nós os demais estados.

Russell e Norvig (2013, p. 109) nos apresentam o algoritmo geral chamado BUSCA-EM-ÁRVORE, sua estrutura é a base para os outros algoritmos de busca, que tendem a variar de acordo com a estratégia de busca utilizada. A fim de evitar caminhos redundantes o algoritmo de BUSCA-EM-ÁRVORE é expandido para BUSCA-EM-GRAFO conforme Figura 1.

Figura 1 - Algoritmo geral de busca em árvore e busca em grafo



função BUSCA-EM-ÁRVORE(*problema*) **retorna** uma solução ou falha
inicializar a borda utilizando o estado inicial do *problema*
repita
se *borda vazia* **então retornar** falha
escolher um nó folha e o remover da borda
se o nó contém um estado objetivo **então retornar** a solução correspondente
expandir o nó escolhido, adicionando os nós resultantes à borda

função BUSCA-EM-GRAFO(*problema*) **retorna** uma solução ou falha
inicializar a borda utilizando o estado inicial do *problema*
inicializar o conjunto explorado tornando-o vazio
repita
se *borda vazia* **então retornar** falha
escolher um nó folha e o remover da borda
se o nó contiver um estado objetivo **então retornar** a solução correspondente
adicionar o nó ao conjunto explorado
expandir o nó escolhido, adicionando os nós resultantes à borda
apenas se não estiver na borda ou no conjunto explorado

Fonte: Russell e Norvig (2013).

2.2.1 Estratégias de Busca sem Informação

Essa estratégia de busca não possui informação adicional sobre os estados, apenas a definição do problema, Russel e Norvig (2013) nos descrevem conforme apresentados a seguir os algoritmos básicos que a utilizam, sendo alguns deles: busca em largura, em profundidade, profundidade limitada e de aprofundamento iterativo.

2.2.1.1 Busca em largura

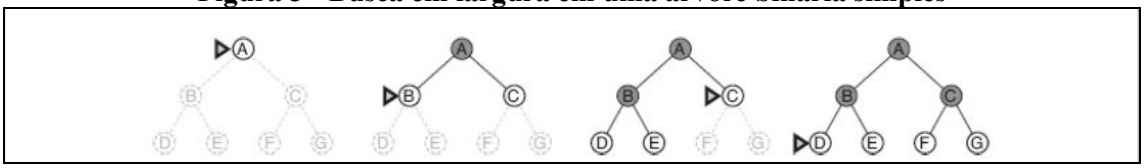
Expande-se primeiro o nó raiz seguindo com a expansão de todos seus sucessores, em seguida os sucessores destes, e assim segue-se. Até o retorno com a solução, a expansão dos nós na profundidade atual ocorrerá com todos, somente depois disso e se necessário, expandirá os nós do nível seguinte. As figuras 2 e 3 apresentam, respectivamente, o pseudocódigo e o progresso da busca em uma árvore binária, onde o próximo nó a ser expandido possui um marcador.

Figura 2 - Pseudocódigo busca em largura em árvore binária simples

```
função BUSCA-EM-LARGURA(problema) retorna uma solução ou falha
  nó ← um nó com ESTADO = problema.ESTADO-INICIAL, CUSTO-DE-CAMINHO = 0
  se problema.TESTE-DE-OBJETIVO(nó.ESTADO) senão retorne SOLUÇÃO(nó),
  borda ← uma fila FIFO com nó como elemento único
  explorado ← conjunto vazio
  repita
    se VAZIO?(borda), então retorne falha
    nó ← POP(borda) / * escolhe o nó mais raso na borda */
    adicione nó.ESTADO para explorado
    para cada ação em problema.AÇÕES(nó.ESTADO) faça
      filho ← NÓ-FILHO(problema, nó, ação),
      se (filho.ESTADO)não está em explorado ou borda então
        se problema.TESTE-DE-OBJETIVO(filho.ESTADO) então retorne SOLUÇÃO(filho)
        borda ← INSIRA(filho, borda)
```

Fonte: Russell e Norvig (2013).

Figura 3 - Busca em largura em uma árvore binária simples



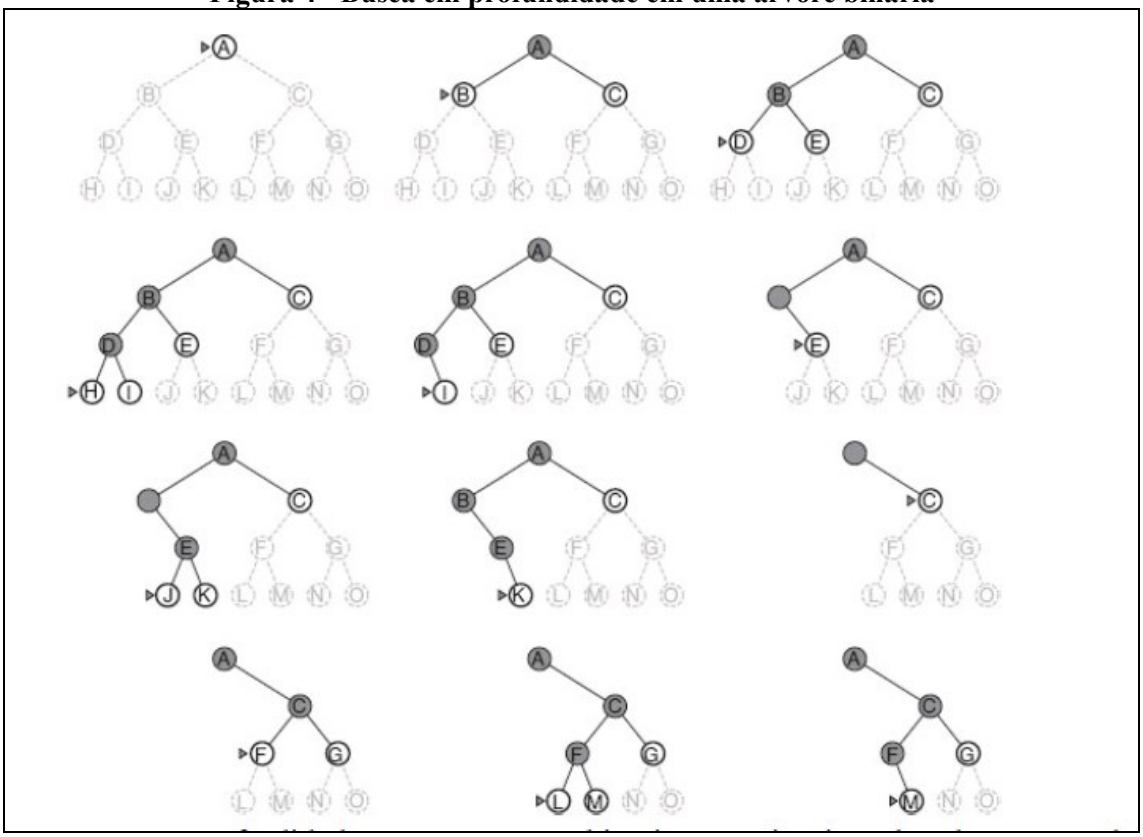
Fonte: Russell e Norvig (2013).

2.2.1.2 Busca em profundidade e com profundidade limitada

A busca em profundidade é caracterizada por sempre expandir o nó mais profundo na borda atual da árvore de busca, prosseguindo até os nós que não possuem sucessores, nível de maior profundidade da árvore. Conforme estes nós se expandem, retiram-se eles da borda, e a busca regressa ao mais profundo nó que ainda não teve seus filhos explorados. (RUSSELL; NORVIG, 2013, p. 119-120).

A fim de evitar o problema dos caminhos infinitos a busca em profundidade normalmente tem seus níveis limitados, ou seja, a profundidade da busca é limitada. A Figura 4 apresenta a busca em uma árvore binária onde a região ainda não explorada mostra-se em cinza-claro. Os nós no terceiro nível de profundidade não têm sucessores e o nó objetivo é exclusivamente o M. (RUSSELL; NORVIG, 2013, p. 120).

Figura 4 - Busca em profundidade em uma árvore binária



Fonte: Russell e Norvig (2013).

2.2.1.3 Busca de aprofundamento iterativo

Conforme exposto por Russell e Norvig (2013, p. 123), é utilizada combinada com a busca em profundidade em árvore, que localiza o mais adequado limite de profundidade. Aumenta-se progressivamente o limite de profundidade, inicia com 0, passa para 1, para 2 e assim segue até o encontro de um objetivo. Tem-se uma combinação da busca em profundidade e busca em largura. Esta abordagem é especialmente útil em cenários onde existe um tempo limitado para realizar as operações. No xadrez, a modalidade Blitz, estabelece um tempo de no máximo 15 minutos para cada jogador durante a partida. Assim, o tempo para decidir por um movimento é bastante reduzido (RUSSELL; NORVIG, 2013, p. 123).

A Figura 5 do algoritmo de busca de aprofundamento iterativo aplica a busca em profundidade limitada com crescentes limites, terminando ao encontrar a solução ou na inexistência desta devolvendo falha.

Figura 5 - Algoritmo de busca de aprofundamento iterativo

```
função BUSCA-DE-APROFUNDAMENTO-ITERATIVO(problema) retorna uma solução ou
 falha
para profundidade = 0 até ∞ faça
resultado ← BUSCA-EM-PROFUNDIDADE-LIMITADA(problema, profundidade)
se resultado ≠ corte então retornar resultado
```

Fonte: Russell e Norvig (2013).

2.2.2 Estratégias de Busca com Informação

Conforme apresentam Russell e Norvig (2013, p. 127) a busca com informação trata-se de uma estratégia que além da definição do problema utiliza conhecimento de um problema específico, podendo ser mais eficaz no encontro da solução do que a busca sem informação.

A abordagem geral chama-se busca de melhor escolha. Trata-se de uma instância da BUSCA-EM-ÁRVORE em seu algoritmo geral. Seleciona-se determinado nó para expansão tomando como base o resultado de uma função de avaliação, *f(n)*. O nó com menor resultado na avaliação é o primeiro a ser expandido, pois a função de avaliação é tratada como uma estimativa de custo (RUSSELL; NORVIG, 2013, p. 119-120).

Russell e Norvig (2013, p. 127) de forma essencial complementam que ao escolher a função de avaliação, determina-se a estratégia de busca. Normalmente os algoritmos de melhor escolha em árvore implementam uma função heurística como componente da função de avaliação.

A função heurística é responsável por retornar o "menor custo estimado do caminho do estado n para um estado objetivo" (RUSSELL; NORVIG, 2013, p. 127), transmitindo assim conhecimento adicional ao algoritmo de busca.

2.2.2.1 Busca gulosa de melhor escolha

A busca gulosa de melhor escolha avalia os nós somente utilizando a função heurística. Visando alcançar de modo rápido uma solução, procura expandir o nó que se encontra mais próximo do estado objetivo (RUSSELL; NORVIG, 2013, p. 127).

2.2.2.2 Busca com ações não determinísticas

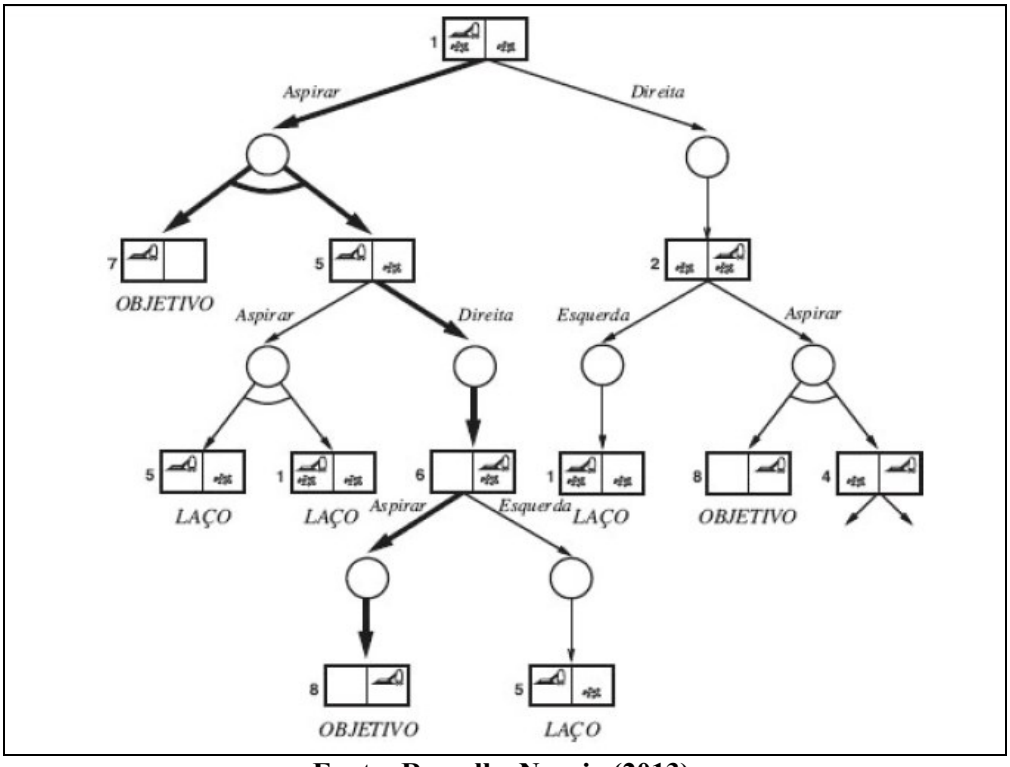
Quando se está diante de ambientes não determinísticos, ou seja, que não são previsíveis apenas com base no estado atual e ações executadas pelo agente, a solução não é mais apenas uma sequência, mas um plano de qual ação realizar a partir das percepções recebidas (RUSSELL; NORVIG, 2013, p.172-173).

2.2.2.3 Árvores de busca E-OU

Os nós em um ambiente determinístico, ramificação apresentada pelas escolhas do agente em cada estado, são chamados de nós OU. Já em um ambiente não determinístico, a ramificação inclui os resultados das escolhas do ambiente para as ações, esses nós chamam-se de E (RUSSELL; NORVIG, 2013, p. 174).

Russell e Norvig (2013, p. 174-175) ilustram, conforme a Figura 6, um exemplo em um mundo de um aspirador de pó defeituoso, cujo agente aspirador escolhe entre as ações mover-se para a direita ou esquerda ou aspirar. Os nós de estado são os nós OU, o estado em que as duas áreas estão limpas representa o estado objetivo. Todos os resultados são tratados nos nós E representados como círculos. Em linhas negritas nos apresentam a solução.

Figura 6 - Árvore de busca E-OU para mundo de aspirador de pó



Fonte: Russell e Norvig (2013).

2.3 FUNCIONAMENTOS DAS *ENGINES* DE XADREZ

A função de avaliação utilizada nas *engines* de xadrez trata-se de uma função que aplica conhecimento heurístico para atribuir valor a uma dada posição de jogo. Quanto maiores as chances de vitória com aquela posição maior seu valor, tendo como linha para vitória o valor 1, para empate o valor 0 e -1 para derrota. Utilizam como parte mais influente da avaliação o valor do balanço material (valores das peças de cada lado) adicionando outras considerações (EVALUATION, 2019).

Conforme exposto por Scarpazza (2015) tomando como exemplo de *engine* o *Stockfish*, lançado em 2004, desenvolvido e disponibilizado em código aberto por Marco Costalba, Joona Kiiski, Gary Linscott, Tord Romstad, Stéphane Nicolet, Stefan Geschwentner e Joost VandeVondele, com muitas contribuições de uma comunidade de desenvolvedores de código aberto, o mecanismo de xadrez leva em consideração para decidir a qualidade da jogada fatores que os humanos também utilizam, tais como:

- Material: Possuir o maior número de peças é melhor, atribuindo valor a cada peça, ex: Peão = 1, Cavalo e Bispo = 3, Torre = 5, e Dama = 9.
- Colocação geral da peça: É preferível controlar as casas centrais do tabuleiro, proteger suas peças que controlam casas importantes assim como atacar as peças inimigas nessa situação.
- Colocação de peças específicas:
    - Cavalos posicionados no centro do tabuleiro protegidos por peão.
    - Bispos controlando as principais diagonais.
    - Rei seguro em casas difíceis de receber xeque-mate, ou em casas com controle das casas ao seu redor.
    - Formação de peões: duplicados, triplos e isolados são formações ruins, peões passados protegidos são bons.

Sabendo agora avaliar as posições, decide-se qual jogada realizar. O *Stockfish* não avalia todo o caminho até o xeque-mate, mas busca com grande nível de profundidade, geralmente mais de 30 nós. Com base em cada estágio do jogo utiliza heurísticas para qualificar os movimentos disponíveis. Isso é possível devido à construção inteligente de uma árvore de profundidade cada vez maior, onde durante a busca calcula o valor dos movimentos e utiliza também técnicas para poda e reconhecimento de padrões (SCARPAZZA, 2015).

Dentre as técnicas utilizadas destacam-se a poda alfa-beta, uma espécie de poda direta chamada *razoring*, e heurísticas de abertura: preferência pelo desenvolvimento de

cavalos e bispos, não mover a dama cedo, realizar o roque, controlar o centro do tabuleiro. Embora o *Stockfish* não adote essa prática, algumas *engines* realizam as jogadas iniciais com movimentos de livros de abertura de xadrez ao invés de realizar cálculos (SCARPAZZA, 2015).

Scarpazza (2015) cita exemplos de heurísticas utilizadas no meio do jogo pelo *Stockfish*, nessa fase é comum os motores ultrapassarem os adversários humanos, a seguir algumas dessas heurísticas:

- Deixar o adversário em situações forçadas a realizar jogadas ruins.
- Ocupar rapidamente as posições ideais para suas peças.
- Atacar com branco tomando iniciativa.
- Defender com preto até a oportunidade de contra-atacar.
- Ocupar rapidamente as posições ideais para suas peças.

Na fase final, o jogo já está resolvido, o *Stockfish* compara a posição de suas peças com uma tabela de possibilidades de posições (*6-man Syzygy Tablebases*) ou utiliza, além disso, heurísticas específicas para cada final de jogo, como relacionadas aos peões passados e movimentos de empate (SCARPAZZA, 2015).

2.4 FORMALIZAÇÕES DAS HEURÍSTICAS NO XADREZ

Conforme estudo realizado por Feitosa *et al.*, (2007) sobre os jogos não de azar, jogos heurísticos como xadrez, observou-se que o ensino normalmente é realizado através de aulas expositivas, fazendo com que os alunos conheçam as regras básicas, porém os conceitos de táticas derivadas das vantagens material, espacial e temporal que podem acelerar a conquista de uma partida são tratadas de maneira subjetiva sem formalismo. Um exemplo apontado por Feitosa *et al.*, (2007), é o fato de não existir um critério claro para os aprendizes de quando uma partida está em sua fase inicial, ou qual é o centro do tabuleiro em cada fase do jogo, e como saber se uma determinada região do tabuleiro está sob controle do jogador.

A abordagem didática pedagógica também baseada na análise de registros de partidas e participação em competições não encaminha o aprendiz a compreender claramente o que são heurísticas e como formalizar suas escolhas de movimentos no andamento das partidas (FEITOSA *et al*., 2007).

Devido à falta de linguagens e ferramentas para especialistas, instrutores e aprendizes, Feitosa *et al*., (2007) desenvolve conceitos e especifica uma linguagem para

definir heurísticas de jogos, chamada DHJOG. Programa um protótipo de ferramenta de autoria que permite a validação da linguagem DHJOG em escolas de xadrez.

A linguagem DHJOG é constituída por seis níveis de expressividade desde iniciante até especialista, cada nível representa uma faixa de conhecimento heurístico, a interface se adapta ao nível e permite a definição visual de regiões, criação de diagramas de transições e etapas, geração manual e automática de código, possibilitando benefícios diretos para o ensino e aprendizagem do xadrez.

A importância da formalização através de ferramentas expostas e desenvolvidas por Feitosa *et al*., (2007) serviu de base e foi ratificada pelo trabalho de Bueno *et al*., (2008) , em que incorpora à ferramenta de autoria métodos matemáticos e estatísticos como a correlação de Kendall, a fim de auxiliar aprendizes e usuários a anteverem o resultado de suas heurísticas em situações distintas de jogo. A interface faz uso dos conceitos de múltiplas representações externas dos esquemas táticos. O método proposto visa à análise de esquemas táticos em xadrez com finalidade educacional.

2.5 BUSCA COMPETITIVA

Conforme definido por Russell e Norvig (2013, p. 204), os jogos originam problemas de busca competitiva, pois temos um ambiente competitivo onde os objetivos de cada agente conflitam-se. Para compreender essa busca os autores consideram a definição de um movimento ótimo, um algoritmo para descobri-lo aliado a utilização da poda e funções de avaliação de heurísticas.

Define-se MAX e MIN como sendo cada um dos jogadores onde MAX é o primeiro a jogar e MIN o segundo. Russell e Norvig (2013, p. 205) definem um jogo como um problema de busca que contém os seguintes elementos:

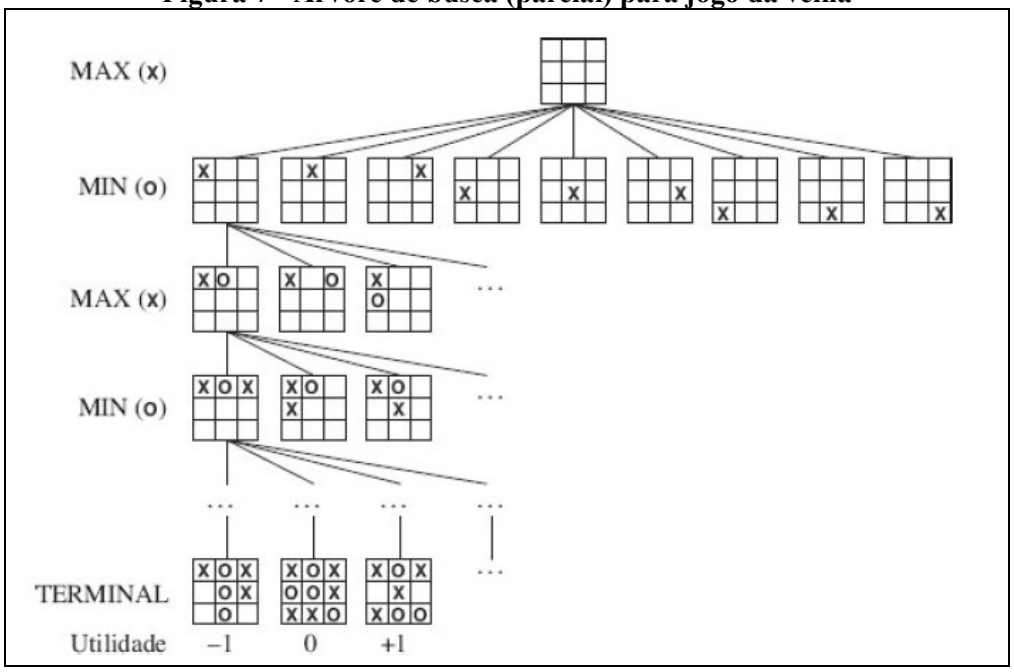
- *S0*: estado inicial, início do jogo.
- JOGADORES(*s*): qual jogador move-se em cada estado.
- AÇÕES(*s*): conjunto de movimentos válidos em dado estado.
- RESULTADO(*s, a*): modelo de transição, responsável pela definição do resultado de um movimento.
- TESTE DE TÉRMINO(*s*): verdadeiro caso o jogo tenha terminado, do contrário falso.
- UTILIDADE(*s, p*): função utilidade responsável por definir valor numérico para estado terminal *s* de um jogo por um jogador *p.* Exemplificando no xadrez vitória com

valor 1, empate com valor ½ ou 0, derrota com valor -1 (RUSSELL; NORVIG, 2013, p. 205).

A árvore de jogo para dado jogo é definida pelo estado inicial, pela função de AÇÕES e pela função de RESULTADO. Os nós representam os estados, as bordas os movimentos, e o número em cada nó de folha referem-se ao valor de utilidade do estado terminal para MAX. Valores de utilidade altos são desejáveis para MAX e indesejáveis para MIN (RUSSELL; NORVIG, 2013, p. 206).

Russell e Norvig (2013, p. 206) exemplificam com uma árvore de busca para o jogo da velha, Figura 7, onde o nó superior corresponde ao estado inicial, MAX começa colocando um X em uma área vazia, a árvore de busca fornece os movimentos alternados por MIN(x) e MAX(o) até os estados terminais, onde de acordo com as regras do jogo pode se atribuir utilidades.

Figura 7 - Árvore de busca (parcial) para jogo da velha



Fonte: Russell e Norvig (2013).

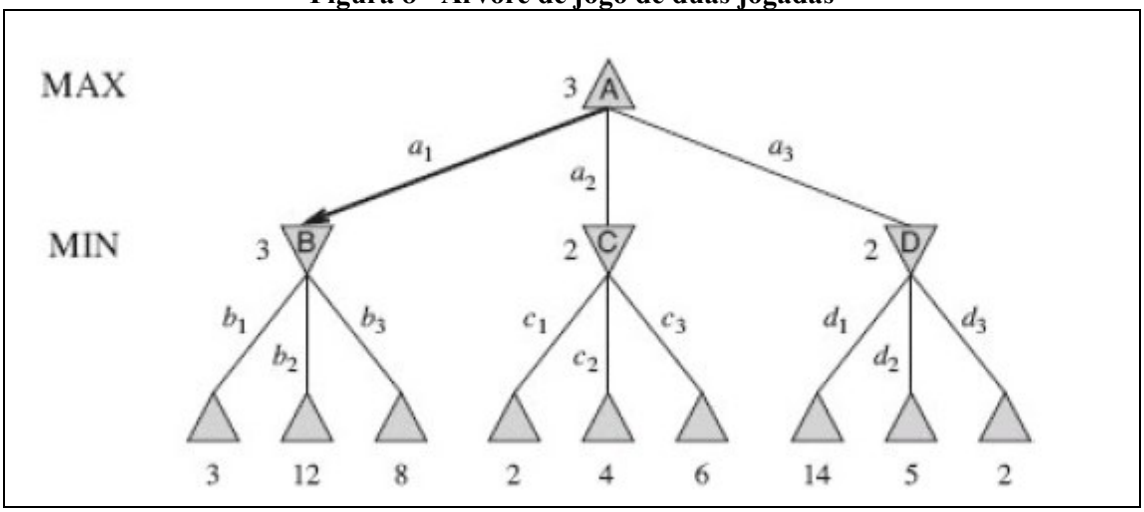
2.5.1 Decisões Ótimas Em Jogos

MAX necessita formar uma estratégia, que especifique seus movimentos no estado inicial e seus movimentos nos estados resultantes para cada movimento possível respondido por MIN, depois MAX preocupa-se com os movimentos a serem realizados como resposta aos movimentos possíveis de MIN, e assim segue (RUSSELL; NORVIG, 2013, p. 207).

Russell e Norvig (2013, p. 207) comparam ao algoritmo de busca E-OU, onde se tem MAX no lugar de OU e MIN equivale a E. Os autores utilizam ainda um jogo de duas jogadas para exemplificar, Figura 8, onde os movimentos possíveis para MAX são *a1*, *a2*, *a3*. As respostas de MIN possíveis para *a1* são *b1*, *b2*, *b3* e assim sucessivamente.

A estratégia ótima pode ser determinada pelo valor MINIMAX de cada nó (utilidade para MAX de se encontrar naquele estado), na Figura 8, os nós terminais possuem os valores de utilidade para MAX, portanto a melhor jogada para MAX é *a1*, pois leva ao estado com maior valor MINIMAX e a melhor jogada em resposta para MIN é *b1*, pois leva ao estado com menor valor MINIMAX. Considera-se a decisão ótima de MIN, resultando no valor MINIMAX 3, pois se MIN não responder de forma ótima, o valor utilidade será maior e ainda mais vantajoso será para MAX.

Figura 8 - Árvore de jogo de duas jogadas



Fonte: Russell e Norvig (2013).

2.5.2 O Algoritmo MINIMAX

O Algoritmo MINIMAX foi desenvolvido pelo matemático John Von Neumann em 1928 (NEUMANN, 1928). Realiza o cálculo de qual decisão tomar a partir do estado atual. Utiliza recursão simples dos valores MINIMAX de cada estado descendente, implementando os cálculos das funções definidas. Após alcançar as folhas da árvore, os valores MINIMAX retornam de volta pela árvore com a recursão.

Conforme o algoritmo para calcular decisões MINIMAX apresentado na Figura 9 por Russell e Norvig (2013, p. 208), retorna a melhor ação possível, com melhor utilidade, supondo que o oponente joga para minimizar a utilidade. VALOR-MAX e VALOR-MIN são

as funções que determinam o valor de propagação de volta de um estado após percorrerem toda a árvore de jogo até as folhas. Onde arg $\max_a \in s$ *f(a)*calcula o elemento *a* do conjunto *S* que possui o maior valor de *f(a).*

Figura 9 - Algoritmo para calcular decisões MINIMAX

```
função DECISÃO-MINIMAX(estado) retorna uma ação
  retornar arg maxa∈AçõES(s) VALOR-MIN(RESULTADO(estado, a))

função VALOR-MAX(estado) retorna um valor de utilidade
  se TESTE TERMINAL (estado) então retornar UTILIDADE(estado)
  v ← − ∞
  para cada a em AÇÕES(estado) faça
    v ← MAX(v,VALOR-MIN(RESULTADO(s,a)))
  retornar v

função VALOR-MIN(estado) retorna um valor de utilidade
  se TESTE-TERMINAL(estado) então retornar UTILIDADE(estado)
  v ← − ∞
  para cada a em AÇÕES(estado) faça
    v ← MIN(v,VALOR-MAX(RESULTADO(s, a)))
  retornar v
```

Fonte: Russell e Norvig (2013).

2.5.3 Poda Alfa-Beta

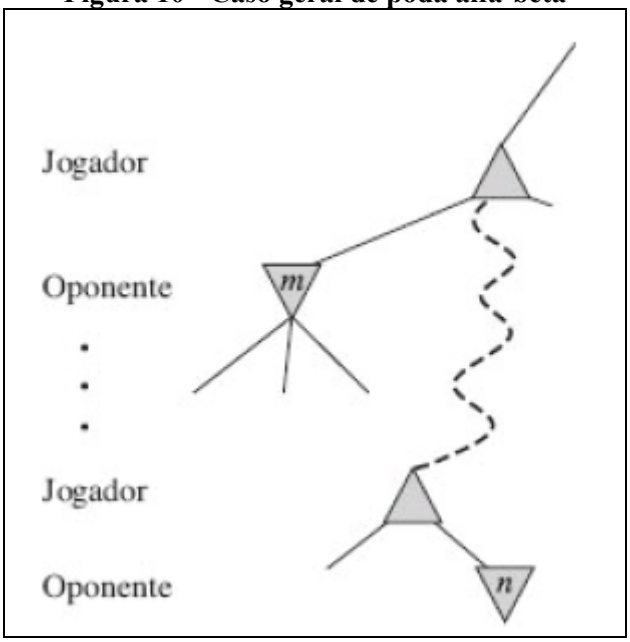
A poda alfa-beta permite calcular a decisão MINIMAX sem precisar examinar todos os nós na árvore de jogo (RUSSELL; NORVIG, 2013, p. 210).

Por meio da Figura 8, já apresentada, é possível perceber que a primeira folha sob B tem o valor 3 que é menor que a segunda e terceira folha de B, portanto MIN realizará o movimento de valor 3. Em C, a primeira folha apresenta o valor 2, então MAX jamais a escolherá dado que em B terá valor 3, assim não há necessidade de examinar as outras folhas de C, ocorrerá a poda. Com relação à D após examinar a primeira e segunda folha por possuírem valores melhores que 3, a terceira folha terá valor menor, terá valor 2, dessa forma a melhor decisão para MAX será o movimento para B (RUSSELL; NORVIG, 2013).

Devido à possibilidade de aplicar a poda alfa-beta em árvores com qualquer nível de profundidade, além de podar folhas consegue-se também podar sub-árvores inteiramente. Se o jogador possui opção de movimento até um nó melhor que determinado nó *n*, seja pai de *n* ou

em outro nível superior, na partida jamais será realizado o movimento até *n*. Conhecendo *n* por examinar alguns de seus filhos pode-se então podá-lo. A Figura 10 ilustra o caso em que *m* é melhor que *n* para o jogador, portanto não se chega a *n* (RUSSELL; NORVIG, 2013, p. 212).

Figura 10 - Caso geral de poda alfa-beta



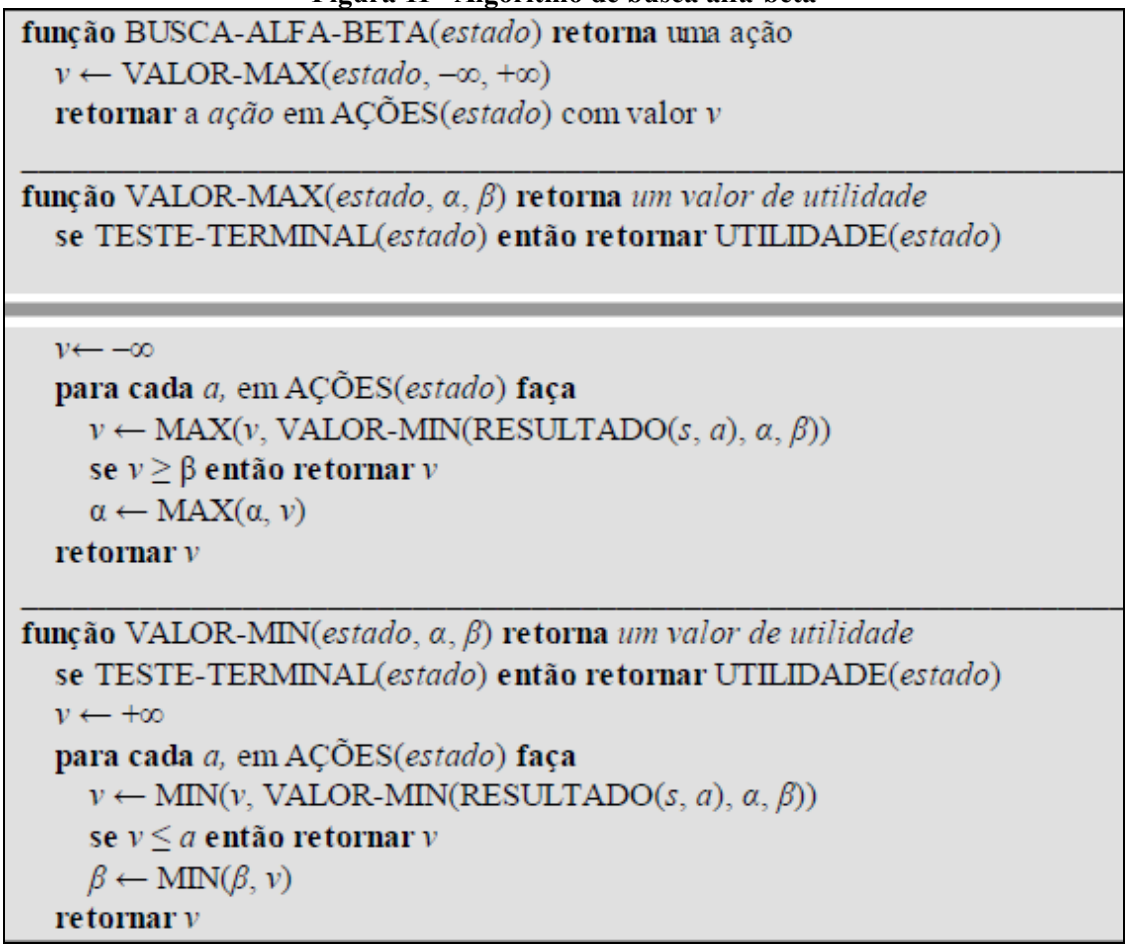
Fonte: Russell e Norvig (2013).

O nome poda alfa-beta origina-se de dois parâmetros (RUSSELL, NORVIG, 2013).

1. α: O maior valor encontrado atualmente para MAX ao longo do caminho, valor da melhor escolha para MAX.
2. β: O menor valor encontrado atualmente para MIN ao longo do caminho, valor da melhor escolha para MIN.

Os valores de alfa e beta são atualizados à medida que a busca ocorre, ao descobrir que o valor atual de um nó é menor que o valor atual de alfa para MAX ou maior que o valor atual de beta para MIN, encerra a recursão e poda as ramificações. A Figura 11 apresenta o algoritmo completo (RUSSELL; NORVIG, 2013, p. 212).

Figura 11 - Algoritmo de busca alfa-beta



```
função BUSCA-ALFA-BETA(estado) retorna uma ação
  v ← VALOR-MAX(estado, −∞, +∞)
  retornar a ação em AÇÕES(estado) com valor v

função VALOR-MAX(estado, α, β) retorna um valor de utilidade
  se TESTE-TERMINAL(estado) então retornar UTILIDADE(estado)

  v ← −∞
  para cada a, em AÇÕES(estado) faça
    v ← MAX(v, VALOR-MIN(RESULTADO(s, a), α, β))
    se v ≥ β então retornar v
    α ← MAX(α, v)
  retornar v

função VALOR-MIN(estado, α, β) retorna um valor de utilidade
  se TESTE-TERMINAL(estado) então retornar UTILIDADE(estado)
  v ← +∞
  para cada a, em AÇÕES(estado) faça
    v ← MIN(v, VALOR-MIN(RESULTADO(s, a), α, β))
    se v ≤ a então retornar v
    β ← MIN(β, v)
  retornar v
```

Fonte: Russell e Norvig (2013).

3 MATERIAIS E MÉTODO

Este capítulo descreve os materiais utilizados na solução do problema em questão, e os passos metodológicos para alcançar os objetivos propostos.

3.1 MATERIAIS

A seguir são apresentadas as principais ferramentas e tecnologias utilizadas para estudo e desenvolvimento da aplicação:

- C#™ 7.3 e Java™ 1.8: Linguagens de programação.
- *Notebook* fabricante ASUSTek Computer Inc., modelo X556URK, sistema operacional *Windows 10 Home Single Language*, 8GB de memória RAM, processador Intel Core i5 – 7200U, 2.5 GHz. Maiores detalhes do equipamento usado para teste estão descritos no apêndice B.
- *Visual Studio 2019 Community Edition*: ambiente utilizado para migração do código da *engine* Java de xadrez para C#.
- *NetBeans IDE 8.0.1*: ambiente utilizado para programação das classes da *engine* de xadrez Java.
- *Logic Crazy Chess*: canal no *YouTube* criado por Jonathan Warkentin, com vídeos tutoriais de programação, *playlist* com a programação de sua *Advanced Java Chess Engine.*
- *Advanced Java Chess Engine*: desenvolvida por Jonathan Warketin com o código fonte das classes, compartilhado pelo autor em repositório do OneDrive.
- *Arena Chess GUI 3.5.1 for Windows*: interface gratuita para *engines* de xadrez, com versão também para Linux.
- *Stockfish Chess v.10 for Windows*: uma das principais *engines* de xadrez existentes, de código aberto possuiu versões para *Windows, Mac, Android e Linux*.
- *Microsoft Office Excel 2007*: Aplicativo de criação de planilhas desenvolvido pela *Microsoft,* utilizado nesse trabalho para realização de cálculos e geração de gráficos.

3.2 MÉTODO

Primeiramente foi efetuada a migração do código Java para C# utilizando o ambiente *Visual Studio 2019* versão *Community Edition*. Esta migração envolveu o estudo das classes Java e suas equivalentes em C#. Bem como a criação de classes novas, por não existir equivalente e pela necessidade de uma classe para armazenar variáveis com os tempos de execução e calcular os tempos médios de execução.

Em segundo lugar foram executados testes preliminares a fim de garantir que as duas *engines* apresentavam os mesmos resultados para situações idênticas, gerando uma sequência de movimentos para diferentes profundidades e estados de jogo.

Em terceiro lugar, foram realizados os testes comparativos de desempenho utilizando as mesmas sequências de movimentos anteriormente gerada, testadas nas seguintes condições: (a) primeiro movimento (b) situação de abertura de jogo, (c) situação de meio de jogo e (d) situação de final de jogo. Isto foi necessário, pois o tamanho da árvore de busca, bem como o fator de ramificação, e a quantidade de peças diferem em cada fase do jogo, as profundidades de busca utilizadas foram profundidade 3 e profundidade 5, o suficiente para alcançar os objetivos desse trabalho.

Cada condição de teste foi executada no mínimo 5 vezes, permitindo assim uma amostragem mais adequada para o cálculo do tempo médio de execução de cada método durante a busca. Isto foi feito devido para evitar que alguma suspensão da própria máquina e/ou do sistema operacional, em virtude de outros processos estarem concorrendo mascarasse os resultados. Além disto, para cada condição de teste, o aplicativo era reiniciado e a primeira execução das sequências de teste foi descartada através do comando *zerar*, criado na *interface* com esse propósito a fim de evitar os efeitos da fase de -"*warm-up*"- (aquecimento), onde o programa está sendo alocado, otimizado e a memória *cachê* está sendo preenchida.

A classe Bench foi criada para medir os resultados de desempenho, possuindo constantes nominadas com o nome de cada método utilizado no processo de busca do melhor movimento, relevantes para a análise de desempenho. Estas constantes foram utilizadas como índices para os vetores que serviram de acumuladores dos parâmetros de desempenho considerados (número de chamadas ao método e tempo de execução). A Listagem 1 apresenta a declaração destas constantes:

Listagem 1 - Constantes da classe Bench

```
public const int FLIPBOARD = 0;
public const int KINGSAFE = 1;
public const int RATE_ATTACK = 2;
```

```
public const int RATE_MATERIAL = 3;
public const int RATE_MOVEABLITLY = 4;
public const int RATE_POSITIONAL = 5;
public const int EVALUATE = 6;
public const int ZWSEARCH = 7;
public const int GETFIRSTLEGALMOVE = 8;
public const int PVSEARCH = 9;
```

Fonte: Autoria própria.

Foram criados também vetores para armazenar o número de chamadas de cada método e o tempo total de execução do mesmo. No caso dos métodos recursivos, foram computados o número de recursões acionadas, tanto para o pvSearch como para o zWSearch. A Listagem 2, mostra um exemplo de código utilizado para declarar cada vetor, onde NFUNC representa o número de funções analisadas:

Listagem 2 - Exemplo de declaração dos vetores

```
public static long[] chamadas = new long[NFUNC];
```

Fonte: Autoria própria.

Para poder medir o tempo que cada função levou para ser executada, foram implementados contadores de tempo para cada função, através de um *array* de *Stopwatch* (classe de cronometragem de tempo). No início de cada método alvo da medição, o cronômetro correlato ao método era inicializado e interrompido antes da saída do método para métodos não recursivos. Nos métodos recursivos, o cronômetro era suspenso quando se invocava outro método interno e antes de retornar, e era retomado no início de sua execução.

Conforme apresentado na Listagem 3 o armazenamento dos tempos de execução foi realizado em milissegundos dentro do *array* de *Stopwacth* chamado ms:

Listagem 3 - Array de Stopwatch

```
public static Stopwatch[] ms = new Stopwatch[NFUNC];
```

Fonte: Autoria própria.

Desse modo foi possível calcular o tempo médio levado para execução de cada função, sendo adicionado também *arrays* para armazenar o menor tempo e o maior tempo de

execução e *array* de *StopWatch* para localizar o menor e maior tempo, o código pode ser observado na Listagem 4:

Listagem 4 - Arrays para armazenar e localizar menor e maior tempo

```
public static long[] maiorMs = new long[NFUNC];
public static long[] menorMs = new long[NFUNC];
public static Stopwatch[] msAux = new Stopwatch[NFUNC];
```

Fonte: Autoria própria.

O método bench é o responsável por escrever na tela da console os resultados do número de chamadas de cada método, tempo médio, maior tempo e menor tempo, ao final da execução do programa, assim como responsável por registrar em arquivo .txt externo todos os tempos calculados para cada chamada recursiva de zWSearch e evalute.

Os registros de tempo de execução de zWSearch e evalute gravados nos arquivos .txt foram utilizados para gerar um gráfico de linha, a fim de comparar quantos picos de tempo de execução ocorreram durante o teste. Tais picos podem ser decorrentes de suspensão por tempo compartilhado do sistema operacional, ou por operações de coleta de lixo embutidos pelos compiladores das linguagens, entre outras possibilidades.

4 RESULTADOS

Este capítulo apresenta os principais resultados obtidos com este trabalho de migração de uma *engine* Java para C# utilizando algoritmo MINIMAX com poda alfa-beta.

4.1 ESCOPO DO SISTEMA

As *engines* de xadrez Java e C# utilizam como base o algoritmo MINIMAX com poda alfa-beta, e empregam funções de avaliação para ser capaz de tomar as decisões sobre qual jogada realizar em uma partida de xadrez.

O motor do jogo é uma aplicação console que permite ser integrada com a interface gráfica *Arena Chess GUI*. A migração para linguagem C# foi realizada através da ferramenta *Visual Studio 2019 Community Edition*.

A partida respeita as regras oficiais do jogo, e através da interface gráfica é possível determinar ou retirar o tempo de duração da partida. Três resultados podem ocorrer para ambos os lados: vitória(1), empate(0) ou derrota(-1).

4.2 MODELAGEM DO SISTEMA

As figuras 12 e 13 apresentam os diagramas UML das classes utilizadas no projeto. Os diagramas foram gerados por meio da *IDE NetBeans* utilizando o *plugin EasyUML*.

A classe Orion contém o método principal de inicialização da *engine*.

A classe Bench é a responsável por executar os cálculos de desempenho. Foi criada e adicionada aos projetos com a finalidade de armazenar e apresentar os resultados dos testes de desempenho. Registra os tempos decorridos em cada chamada realizada pelos métodos utilizados na busca.

O estado do tabuleiro atual com a disposição das peças em posição inicial ou de meio de jogo, por exemplo, é controlado pela classe BoardGeneration, enquanto os movimentos permitidos são validados pela classe Moves.

Os comandos do aplicativo *console,* como os utilizados para iniciar o jogo, ler as jogadas realizadas e solicitar a busca do melhor movimento encontram-se disponíveis na classe UCI.

O resultado do melhor movimento é calculado pela classe PrincipalVariation utilizando como função de avaliação as funções implementadas na classe Rating.

Em virtude da falta de espaço e a preferência por não impedir a legibilidade, os métodos das classes BoardGeneration, Moves e PrincipalVariation tiveram seus parâmetros retirados nas figuras 12 e 13, mas estão descritos nos quadros 1, 2, 3 a seguir de forma completa.

Quadro 1 - Métodos da classe BoardGeneration

Classe BoardGeneration
static void initiateStandardChess()
static void initiateChess960()
static void importFEN(String fenString)
static void arrayToBitboards(String[][] chessBoard, long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK)
static long convertStringToBitboard(String Binary)
static void drawArray(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK)
static void flipBoard()
static boolean kingSafe()
static void getChessboardArray(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK)

Fonte: Autoria própria.

Quadro 2 - Métodos da classe Moves

Classe Moves
static long HAndVMoves(int s)
static long DAndAntiDMoves(int s)
static long makeMove(long board, String move, char type)
static long makeMoveCastle(long rookBoard, long kingBoard, String move, char type)
static long makeMoveEP(long board, String move)
static String possibleMovesW(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK, long EP, boolean CWK, boolean CWQ, boolean CBK, boolean CBQ), static String possibleMovesB(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK, long EP, boolean CWK, boolean CWQ, boolean CBK, boolean CBQ)
static String possibleWP(long WP, long BP, long EP)

static String possibleBP(long BP, long WP, long EP)
static String possibleN(long OCCUPIED, long N)
static String possibleB(long OCCUPIED, long B)
static String possibleR(long OCCUPIED, long R)
static String possibleQ(long OCCUPIED, long Q)
static String possibleK(long OCCUPIED, long K)
static String possibleCW(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK, boolean CWK, boolean CWQ)
static String possibleCB(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK, boolean CBK, boolean CBQ)
static long unsafeForBlack(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK)
static long unsafeForWhite(long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK)
static void drawBitboard(long bitBoard)
static void timeExperiment(long WP)
static void tEMethodA(int loopLength, long WP)
static void tEMethodB(int loopLength, long WP)

Fonte: Autoria própria.

Quadro 3 - Métodos da classe PrincipalVariation

Classe PrincipalVariation
static int zWSearch(int beta, long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK, long EP, boolean CWK, boolean CWQ, boolean CBK, boolean CBQ, boolean WhiteToMove, int depth)
static int getFirstLegalMove(String moves, long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK, long EP, boolean CWK, boolean CWQ, boolean CBK, boolean CBQ, boolean WhiteToMove)
static int pvSearch(int alpha, int beta, long WP, long WN, long WB, long WR, long WQ, long WK, long BP, long BN, long BB, long BR, long BQ, long BK, long EP, boolean CWK, boolean CWQ, boolean CBK, boolean CBQ, boolean WhiteToMove, int depth)

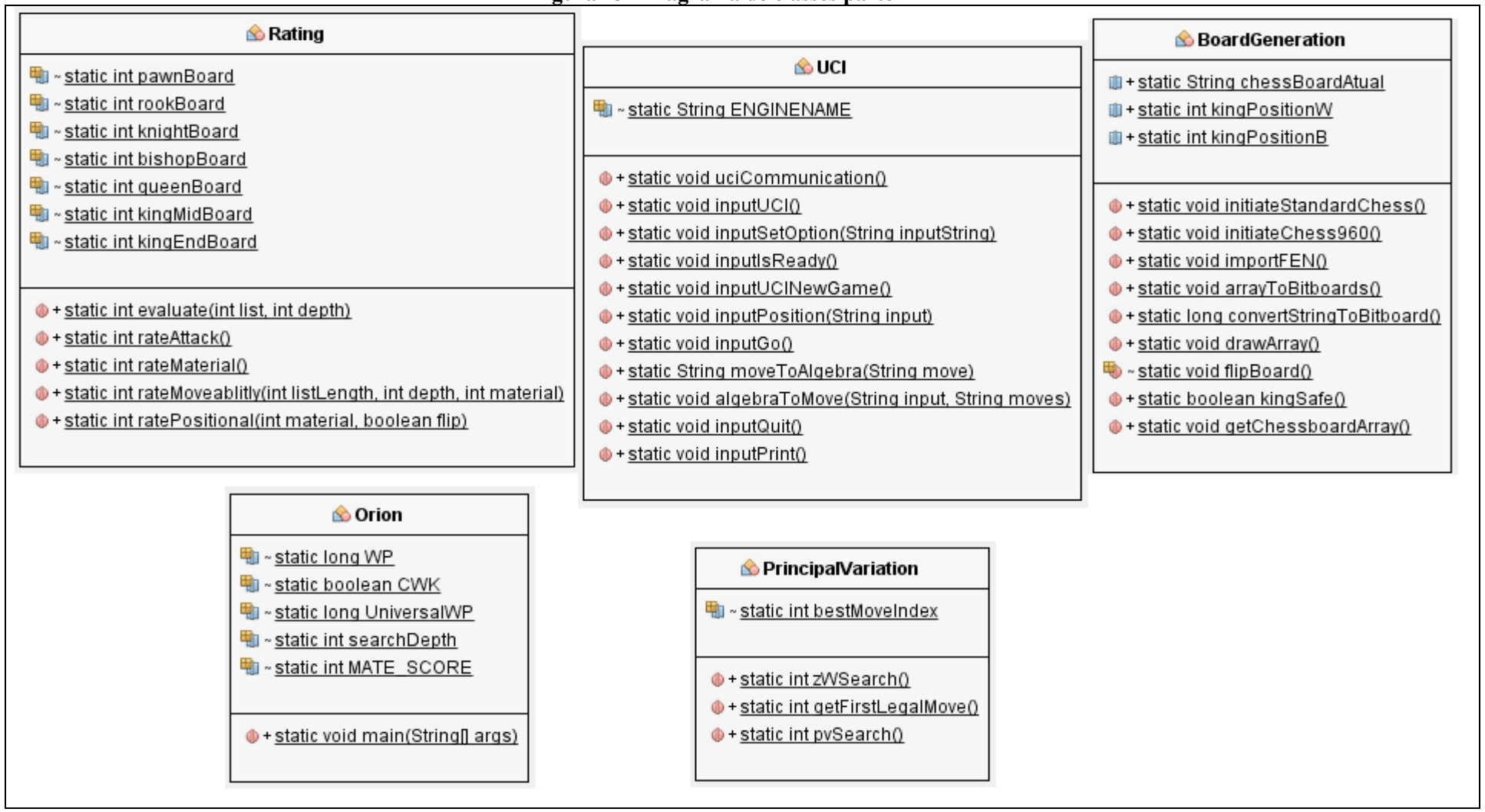
Fonte: Autoria própria.

Figura 12 - Diagrama de classes parte 1



Fonte: Autoria própria.

Figura 13 - Diagrama de classes parte 2



Fonte: Autoria própria.

4.3 APRESENTAÇÃO DO SISTEMA

A *Advanced Engine* em Java nomeada pelo autor em seu projeto como Orion trata-se de aplicativo *.jar* que foi migrada para aplicativo *Windows* de *console* em linguagem C#, nomeado como OrionCSharp. Os códigos fonte da *engine* Java e da *engine* C#, utilizados neste trabalho, encontram-se disponíveis para download nos seguintes repositórios:

1. https://github.com/valdineibucmaier16/Engine_Java
2. https://github.com/valdineibucmaier16/Engine_CSharp

As *engines* foram desenvolvidas dentro de um padrão de protocolo chamado UCI (*universal chess interface* - *interface* universal de xadrez) que permite a integração com interfaces gráficas compatíveis, como o *software Arena Chess GUI* (Figura 14), utilizado nesse trabalho como interface gráfica e utilizado também pelo autor em seu canal no *YouTube*.

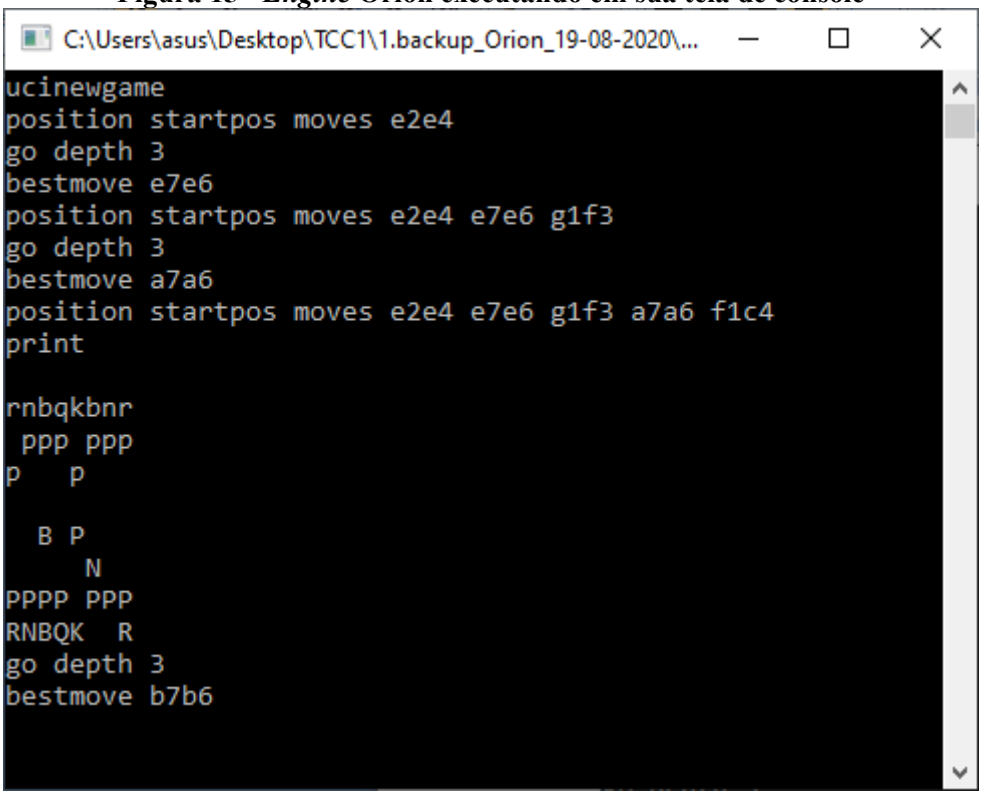
Figura 14 - Tela principal do *software Arena Chess GUI*



Fonte: Autoria própria.

Os comandos interpretados pelo *Arena* desenvolvidos no padrão *UCI* podem ser observados nas classes *UCI* dos projetos Orion e OrionCSharp dentro do método uciComunication(). A Figura 15 apresenta a aplicação sendo executada em sua tela de *console* sem a integração com o *Arena*, dessa forma é possível visualizar os comandos UCI realizados e seus resultados na tela.

Figura 15 - *Engine* Orion executando em sua tela de console



Fonte: Autoria própria.

Conforme podemos observar na Figura 15 o comando *ucinewgame* inicia uma nova partida, position startpos moves e2e4 realiza o primeiro movimento e também a leitura dos movimentos já realizados, no primeiro movimento a peça da casa e2 foi movida para e4, go depth 3 realiza a busca do melhor movimento de resposta calculado na profundidade 3, o melhor movimento calculado é o resultado *bestmove* e será a jogada realizada pela *engine,* os comandos são repetidos permitindo o andamento da partida.

A representação visual do tabuleiro na console é feita através do comando *print* onde cada peça é representada na tela com a inicial de seu nome em inglês, sendo diferenciadas as de cada jogador por formatação maiúscula e minúscula. Para encerrar a partida utiliza-se o comando *quit*.

O projeto Orion em ambas as versões pode ser vinculado ao *Arena*. Além de permitir testar *engines* o *Arena Chess GUI* oferece também funcionalidades como as de jogar com outras *engines* já desenvolvidas, tais como o *Stockfish*, analisar jogos, e também posições. Permite ler bases de partidas salvas no formato *Portable Game Notation* (PGN), importar ou salvar posições de tabuleiro utilizando o formato *Forsyth-Edwards Notation* (FEN), dentre vários outros recursos configuráveis para enxadristas profissionais ou iniciantes no estudo e prática do xadrez.

4.4 IMPLEMENTAÇÃO DO SISTEMA

Nesta seção é apresentada a implementação do sistema por meio do processo de tradução do código Java para C#, procedimentos para integração das *engines* com a interface gráfica *Arena Chess GUI*, testes de validação dos resultados de busca e testes comparativos de desempenho entre as *engines*.

4.4.1 Tradução do Código

Ao iniciar o processo de migração do código entre as linguagens foi pesquisado sobre a existência de s*oftwares* livres ou com versões gratuitas para teste que permitissem realizar a tradução do código das classes Java para C#, de forma a agilizar o processo e simplificar o trabalho.

Não foram encontrados *softwares* que atendessem a expectativa criada, porém destacou-se o aplicativo *web Code Translation* disponível em https://www.carlosag.net/tools/codetranslator/. Este aplicativo permite selecionar a opção de linguagem desejada de Java para C#, ou vice-versa, e ao colar o código de origem em uma caixa de texto ele realiza a tradução. Entretanto, a aplicação não é capaz de identificar os métodos correspondentes na outra linguagem, sendo eficiente para a tradução de código referente à declaração de tipos de variáveis e criação de objetos comuns entre as linguagens, mas que possuem sintaxe diferente ao serem declarados, tomando como exemplo a criação de vetores e matrizes.

Dada a escassez de aplicações, praticamente toda a tradução foi baseada em pesquisas na documentação técnica das linguagens para identificação sobre os métodos correspondentes ou equivalentes. Destaca-se que a maioria dos métodos existe em ambas as linguagens, apresentando inclusive a mesma nomenclatura, mas mesmo nesses casos a

consulta a documentação torna-se necessária pelo fato desses métodos nem sempre apresentarem os retornos no mesmo formato ou nem sempre trabalharem da mesma maneira o cálculo com os parâmetros de entrada.

O Quadro 4 apresenta exemplos de código com os métodos do projeto em Java original na coluna da esquerda e seus equivalentes no projeto em C# na coluna da direita. É informada também a classe do projeto em que o trecho de código foi retirado.

Utilizada na classe Moves, a classe LongExtensions e seus métodos foram criados no código C#. Seus métodos Reverse e NumberOfTrailingZeros foram adaptados do Java para receber variável do tipo ulong como parâmetro. Uma grande facilidade oferecida pelo Java nessa tarefa foi o fato que seus métodos estão escritos em código aberto, sendo possível inspecionar a lógica e adaptá-la para o C#.

Quadro 4 - Código em Java e código correspondente em C#

Código em Java	**Código C#**
Classe BoardGeneration	
int random1=(int)(Math.random()*8);	Random randNum = new Random(); int random1 = ((int)((randNum.Next() * 8)));
if (" ".equals(chessBoard[0][loop])) {counter++;}	if (" ".Equals(chessBoard[0, loop])) {counter++;}
while (fenString.charAt(charIndex) != ' ')	while (fenString.ToCharArray()[charIndex] != ' ')
Binary=Binary.substring(i+1)+"1"+Binary.subst ring(0, i);	Binary = Binary.Substring(i + 1) + "1" + Binary.Substring(0, i);
return Long.parseLong(Binary, 2);	return Convert.ToInt64(Binary, 2);
System.out.println(Arrays.toString(chessBoard[i]));	for (int j = 0; j < 8; j++) Console.Write(chessBoard[i, j]);
if (Character.isUpperCase(chessBoardAtual[r][c]. charAt(0))) { temp=chessBoardAtual[r][c].toLowerCase(); }	if (char.IsUpper(chessBoardAtual[r, c].ToCharArray()[0])){ temp = chessBoardAtual[r, c].ToLower();}
Classe Moves	
(Long.reverse(OCCUPIED) - 2 * Long.reverse(binaryS))	(LongExtensions.Reverse(OCCUPIED) - 2 * LongExtensions.Reverse(binaryS))
(Character.getNumericValue(move.charAt(0))	(char.GetNumericValue (move.ToCharArray()[0])
start=Long.numberOfTrailingZeros	start = LongExtensions.NumberOfTrailingZeros

(FileMasks8[move.charAt(0)- '0']&RankMasks8[1]);	(FileMasks8[move.ToCharArray()[0] - '0'] & RankMasks8[1]);
Classe PrincipalVariation	
if (Character.isDigit(moves.charAt(firstLegalMov e + 3)))	if (Char.IsDigit(moves.ToCharArray()[i + 3]))
Classe UCI	
Scanner input = new Scanner(System.in);	String inputString = Console.ReadLine();
(inputString.startsWith("setoption"))	(inputString.StartsWith("setoption"))
input=input.substring(9).concat(" ");	input = input.Substring(9) + " ";

Fonte: Autoria própria.

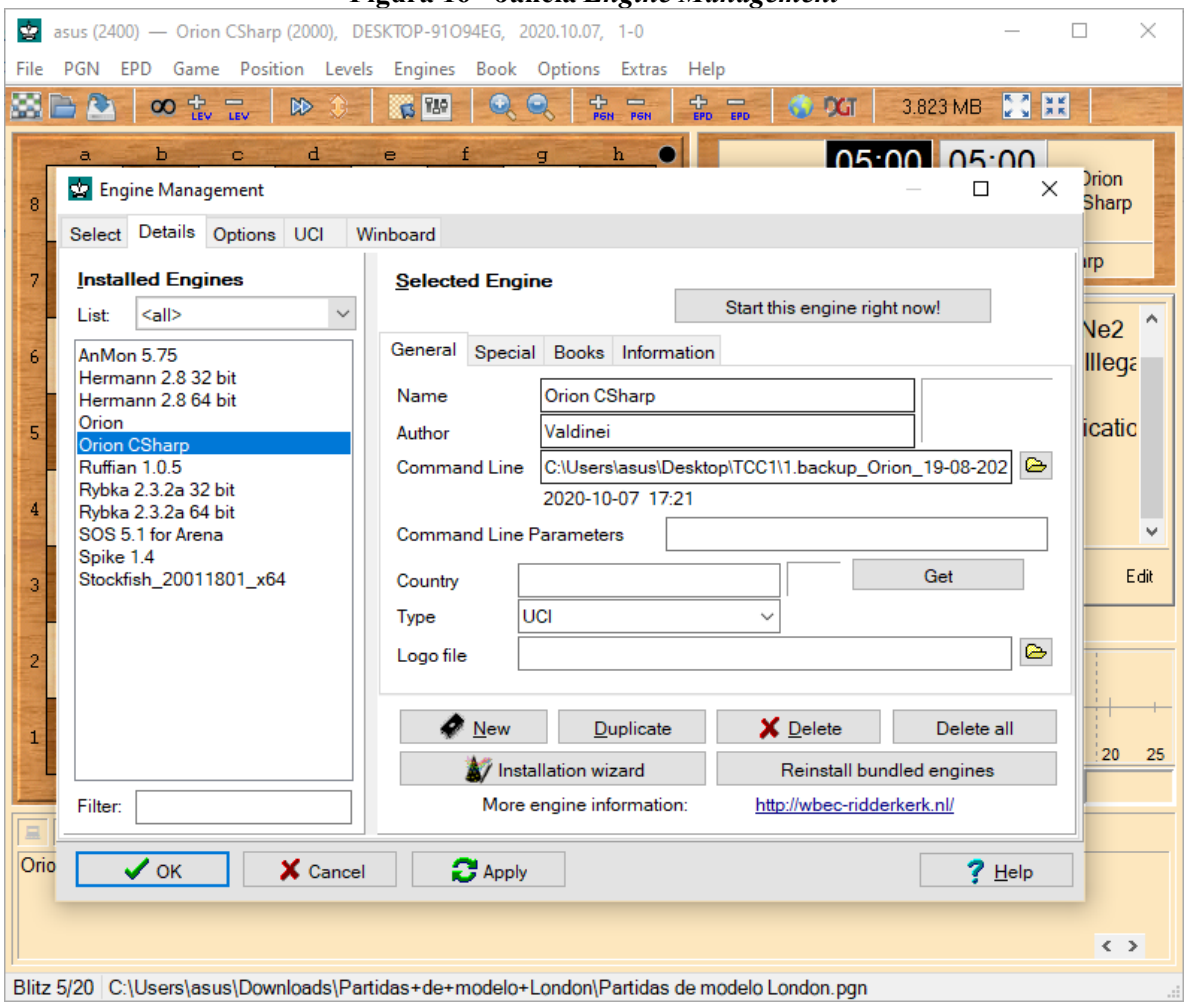
4.4.2 Integração com *Arena*

O projeto Orion em ambas as versões pode ser vinculado à interface gráfica *Arena* realizando as seguintes etapas: na barra de menus inicial clica-se em *Engines*, *Manage...*, abrindo assim a janela *Engine Management.* Esta janela pode ser aberta também através da tecla de atalho F11.

Na aba *Details* da janela *Engine Management,* conforme pode ser observada na Figura 16, em *Selected Engine* encontra-se a aba *General*, em que dev e-se selecionar na caixa de listagem *Type* a opção UCI. O *Arena* comunica-se também com *engines* que utilizam o protocolo *Winboard*.

Para adicionar a nova *engine* a ser gerenciada pelo *Arena* clica-se no botão *New*, e informa-se o caminho do arquivo Orion.jar ou do executável da versão em C#, sendo possível definir um nome para cada uma delas. Finaliza-se a adição da *engine* ao *Arena* clicando no botão OK. Para adicionar a *engine* Orion na outra versão ou outra *engine* já existente repete-se o procedimento.

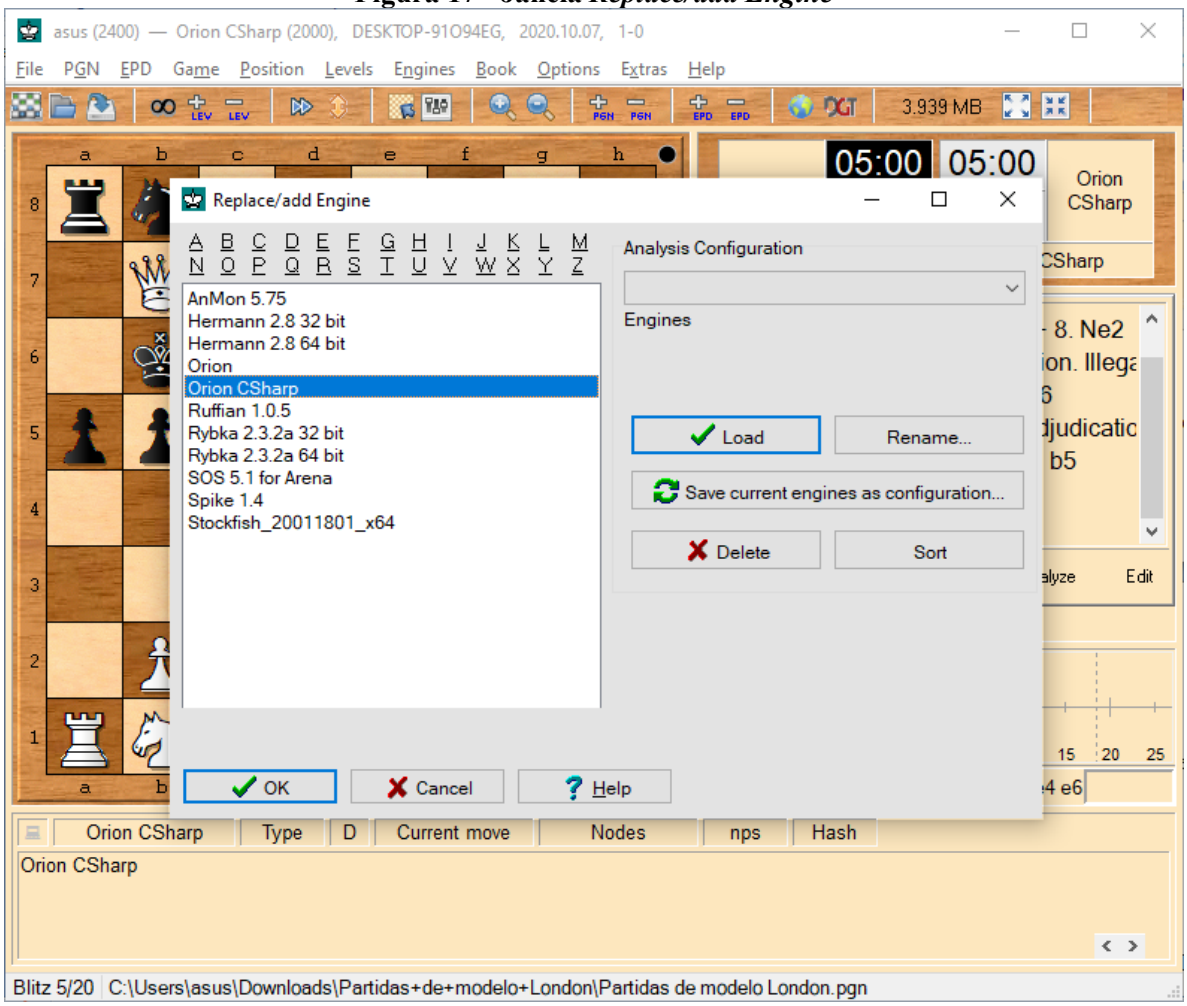
Figura 16 - Janela *Engine Management*



Fonte: Autoria própria.

Uma vez adicionadas as *engines,* para utilizá-las, deve ser feito a sua carga por meio da opção *Engines* da barra de menu, e *Load Engines.* Abre-se então a janela *Replace/add Engine*, apresentada na Figura 17. Seleciona-se a *engine* desejada e a carrega clicando no botão *Load*. Finaliza-se com clique no botão *OK*.

Figura 17 - Janela *Replace/add Engine*



Fonte: Autoria própria.

Um novo jogo pode ser iniciado por meio da opção de menu *File*, em *New* ou utilizando o atalho *control*+N. Para abrir uma partida salva em arquivo no formato PGN, clica-se na opção de menu PGN, seguido de *Open*, ou por meio do atalho *control*+E.

Durantes os testes foram utilizados diferentes estados de jogo, como de abertura e final de jogo, tendo sido necessário importar estados de tabuleiro salvos no formato FEN.

Para importar no *Arena* o estado de determinada posição do tabuleiro, primeiro realiza-se uma cópia no texto da notação FEN para o *clipboard*, em seguida clica-se no menu *Position* e *Get FEN from Clipboard*, ou pressiona-se a tecla de atalho F6.

Para criar as representações FEN de meio de jogo e final de jogo, utilizadas nos testes de validação e desempenho entre as *engines,* foi utilizado o recurso do *Arena Copy FEN to Clipboard*, disponível no menu *Position*.

A notação FEN correspondente a posição inicial de uma partida, na Figura 18 é representada por “rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1”.

A parte “rnbqkbnr/pppppppp/” são as linhas 8 e 7 do tabuleiro compostas pelas peças negras e o nome de cada peça é dado pelas suas iniciais em inglês (“R” de *Rook* para a Torre, “K” de *Knight* para o Cavalo, “B” de *Bishop* para o Bispo, “Q” de *Queen* para a Dama, “K” de *King* para o Rei e “P” de *Pawn* para o Peão) sendo em letras minúsculas para as pretas e maiúsculas para as brancas.

Segue-se com “8/8/8/8/” que representa as linhas 6,5,4 e 3 do tabuleiro que não estão ocupadas por nenhuma peça, e “PPPPPPPP/RNBQKBNR” são as linhas 2 e 1 ocupadas pelas peças negras.

Após exposto a posição das peças a notação “w” é responsável por indicar a vez do jogador, nesse caso “w” de *White* indica que é a vez das brancas, como estamos na posição inicial é natural que as brancas sempre comecem. Seguimos com “KQkq” indicando qual roque os jogadores podem fazer, roque para a ala do Rei ou para a ala da Dama, nesse caso todos os roques estão permitidos para ambos os jogadores.

Os endereços das casas de *en-passant* vêm após a informação do roque, no nosso exemplo está com “-” pois só existirá após uma jogada de peão em que o peão avance duas casas. Por fim segue numeração para informar quantos movimentos existiram desde que a última peça foi capturada ou movimento de peões caso não haja ainda nenhuma captura, temos assim “0”, seguido por “1” que indica o número da jogada, sempre incrementando 1 após a jogada ser concluída com a resposta das pretas.

Figura 18 - Tabuleiro com posição de início de jogo



Fonte: Autoria própria.

Além da posição inicial de jogo, foram utilizadas nos testes a posição "rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ - 2 2" para situações de meio de jogo e a "1r6/3k4/8/3p4/5P2/2p5/P7/4K3 w - - 0 27" para representar uma situação de final de jogo. As posições do tabuleiro correspondentes podem ser observadas nas Figuras 19 e 20.

Figura 19 - Tabuleiro com posição de meio de jogo



Fonte: Autoria própria.

Figura 20 - Tabuleiro com posição de final de jogo



Fonte: Autoria própria.

4.4.3 Testes de Validação

A fim de verificar a corretude do código C#, foram realizados 10 testes na forma de jogadas executadas por um jogador humano como brancas e a *engine* jogando como pretas em diferentes profundidades e fases do jogo. As jogadas foram realizadas com ambas as *engines* integradas ao Arena, facilitando a visualização e registro dos movimentos.

Dado que em todos os testes ambas as *engines* apresentaram a mesma sequência de movimentos como resposta para os realizados pelo jogador humano, provou-se que o código C# está corretamente escrito, podendo-se então avançar para a segunda etapa de testes.

No Arena, a caixa do lado direito do tabuleiro com a aba “Movelist” selecionada apresenta todas as jogadas realizadas. O último movimento escolhido pela *engine* aparece na parte inferior da janela descrito como “Best move”. A seguir, estão descritos alguns dos resultados dos testes de validação apresentados em ambas as *engines*:

O resultado para a situação de primeiro movimento é apresentado na Figura 21 para profundidade de busca 3, sendo esse também o resultado para profundidade 5.

Figura 21 - Resultado primeiro movimento, profundidades 3 e 5



Fonte: Autoria própria.

Para a fase de abertura, o resultado para profundidade 3 é apresentado na Figura 22, e para profundidade 5 na Figura 23.

Figura 22 - Resultado abertura, profundidade 3



Fonte: Autoria própria.

Figura 23 - Resultado abertura, profundidade 5



Fonte: Autoria própria

Para os testes de meio jogo foi utilizada como posição a ilustrada na Figura 19 (FEN: rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ - 2 2) e o resultado para a profundidade 3 encontra-se na Figura 24 e para a profundidade 5 na Figura 25.

Figura 24 - Resultado meio de jogo, profundidade 3



Fonte: Autoria própria

Figura 25 - Resultado meio de jogo, profundidade 5



Fonte: Autoria própria

Para os testes de final de jogo foi utilizada a posição da Figura 20 (FEN: 1r6/3k4/8/3p4/5P2/2p5/P7/4K3 w - - 0 27) e o resultado para a profundidade 3 foi o mesmo na profundidade 5 e pode ser observado na Figura 26.

Figura 26 - Resultado final de jogo, profundidade 3 e 5



Fonte: Autoria própria.

4.4.4 Testes de Desempenho

Para os testes de desempenho foram utilizadas as bases de movimentos geradas como resultado dos testes de validação. Os testes foram realizados nas profundidades de busca 3 e 5 para as situações de primeiro movimento, abertura, meio de jogo e final de jogo. Foram executados nas *engines* diretamente em suas janelas de console, através dos comandos das aplicações, dispensando a necessidade de integração com a interface gráfica.

A sequência de linhas da *console* que caracteriza cada teste é composta pelas entradas, assim como, pelos retornos da aplicação. Os movimentos de resposta calculados pela *engine* para cada jogada do oponente estão descritos nas linhas iniciadas por *bestmove*.

Os comandos utilizados em ambas as aplicações para os testes de desempenho em cada fase de jogo e profundidade de busca é apresentada nas listagens a seguir: primeiro movimento apresenta-se na Listagem 5 para a profundidade de busca 3 e na Listagem 6 para a

profundidade de busca 5. A fase de abertura é apresentada na Listagem 7 para a profundidade de busca 3 e Listagem 8 para a profundidade de busca 5. Para a fase de meio de jogo observa-se a Listagem 9 para profundidade de busca 3 e Listagem 10 para a profundidade de busca 5. Os comandos do teste de final de jogo na profundidade 3 e 5 constam na Listagem 11.

Listagem 5 – Teste de primeiro movimento, profundidade de busca 3

```
ucinewgame
position startpos moves e2e4
go depth 3
bestmove e7e6
ucinewgame
position startpos moves e2e4
go depth 3
bestmove e7e6
zerar
ucinewgame
position startpos moves e2e4
go depth 3
bestmove e7e6
```

Fonte: Autoria própria.

Listagem 6 - Teste de primeiro movimento, profundidade de busca 5

```
ucinewgame
position startpos moves e2e4
go depth 5
bestmove a7a6
ucinewgame
```

Fonte: Autoria própria.

Listagem 7 - Teste de abertura, profundidade de busca 3

```
ucinewgame
position startpos moves e2e4
go depth 3
bestmove e7e6
position startpos moves e2e4 e7e6 g1f3
```

```
go depth 3
bestmove a7a6
position startpos moves e2e4 e7e6 g1f3 a7a6 f1c4
go depth 3
bestmove b7b6
position startpos moves e2e4 e7e6 g1f3 a7a6 f1c4 b7b6 d2d3
go depth 3
bestmove c7c6
ucinewgame
```

Fonte: Autoria própria.

Listagem 8 - Teste de abertura, profundidade de busca 5

```
ucinewgame
position startpos moves e2e4
go depth 5
bestmove a7a6
position startpos moves e2e4 a7a6 g1f3
go depth 5
bestmove b7b6
position startpos moves e2e4 a7a6 g1f3 b7b6 f1c4
go depth 5
bestmove c7c6
position startpos moves e2e4 a7a6 g1f3 b7b6 f1c4 c7c6 d2d3
go depth 5
bestmove d7d6
ucinewgame
```

Fonte: Autoria própria

Listagem 9 - Teste de meio jogo, profundidade de busca 3

```
ucinewgame
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
- 2 2 moves g4e5
go depth 3
bestmove a7a6
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
```

```
- 2 2 moves g4e5 a7a6 a2a4
go depth 3
bestmove g7g6
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
- 2 2 moves g4e5 a7a6 a2a4 g7g6 h1g1
go depth 3
bestmove h8g7
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
- 2 2 moves g4e5 a7a6 a2a4 g7g6 h1g1 h8g7 c3c4
go depth 3
bestmove g7h8
ucinewgame
```

Fonte: Autoria própria.

Listagem 10 - Teste de meio jogo, profundidade de busca 5

```
ucinewgame
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
- 2 2 moves g4e5
go depth 5
bestmove g7g6
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
- 2 2 moves g4e5 g7g6 h1g1
go depth 5
bestmove a7a6
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
- 2 2 moves g4e5 g7g6 h1g1 a7a6 a2a3
go depth 5
bestmove h7h6
position fen rn4rk/pp4pp/1qp5/3np3/4N1NP/2PP4/PP3P2/R3K2R w KQ
- 2 2 moves g4e5 g7g6 h1g1 a7a6 a2a3 h7h6 a1b1
go depth 5
bestmove a6a5
```

Fonte: Autoria própria

Listagem 11 - Teste de final de jogo, profundidade de busca 3 e 5

```
ucinewgame
position fen 1r6/3k4/8/3p4/5P2/2p5/P7/4K3 w - - 0 27 moves
e1f2
go depth 3
bestmove d5d4
position fen 1r6/3k4/8/3p4/5P2/2p5/P7/4K3 w - - 0 27 moves
e1f2 d5d4 a2a4
go depth 3
bestmove d4d3
position fen 1r6/3k4/8/3p4/5P2/2p5/P7/4K3 w - - 0 27 moves
e1f2 d5d4 a2a4 d4d3 f4f5
go depth 3
bestmove c3c2
position fen 1r6/3k4/8/3p4/5P2/2p5/P7/4K3 w - - 0 27 moves
e1f2 d5d4 a2a4 d4d3 f4f5 c3c2 a4a5
go depth 3
bestmove d3d2
ucinewgame
```

Fonte: Autoria própria

O comando *ucinewgame* permite reiniciar a aplicação sem precisar encerrá-la. Desse modo, é possível repetir o teste correspondente o número de vezes desejado. Os resultados individuais são somados com o de seus testes anteriores para os cálculos de desempenho, com exceção dos testes que forem descartados através do comando *zerar*. O resultado dos cálculos é apresentado na janela do console quando a aplicação é encerrada através do comando *quit*.

As medições de maior e menor tempo não foram consideradas para os métodos recursivos zWSearch e pvSearch, em virtude que, a última chamada recursiva sempre terá o menor tempo do que a primeira. Para estes métodos o tempo total da recursão é que fornece um melhor parâmetro de avaliação do desempenho. Todos os tempos foram medidos em milissegundos.

Durante os testes da fase de primeiro movimento para as profundidades de busca 3 e 5 foram descartados os dois primeiros testes, e realizados mais 20 testes. Os resultados podem ser observados na Tabela 3 do Apêndice A e na Tabela 4 do Apêndice A.

Para a fase de abertura com as profundidades de busca 3 e 5 foram descartados os primeiros dois testes e realizados mais 10 testes. Na Tabela 5 do Apêndice A e na Tabela 6 no Apêndice A são apresentados os resultados destes testes.

Ao realizar os testes da fase de meio de jogo para as profundidades de busca 3 descartou-se os dois primeiros testes e foram realizados mais 10 testes. Para a profundidade de busca 5 descartou-se o primeiro teste e foram realizados mais 4 testes.

Na fase de meio de jogo o número total de testes foi menor para a profundidade 5, pois devido à grande quantidade de chamadas dos métodos, ao registrar em arquivo .txt os tempos decorridos para cada execução dos métodos, o código da *engine* Java apresentava erro de memória observado na Listagem 12. Desse modo o número de testes foi reduzido à mesma quantidade na *engine* C# para permitir a comparação. Os resultados são apresentados na Tabela 7 do Apêndice A e Tabela 8 do Apêndice A.

Listagem 12 - Erro código Java ao gravar no arquivo .txt

```
Exception in thread "main" java.lang.OutOfMemoryError: Java
heap space
```

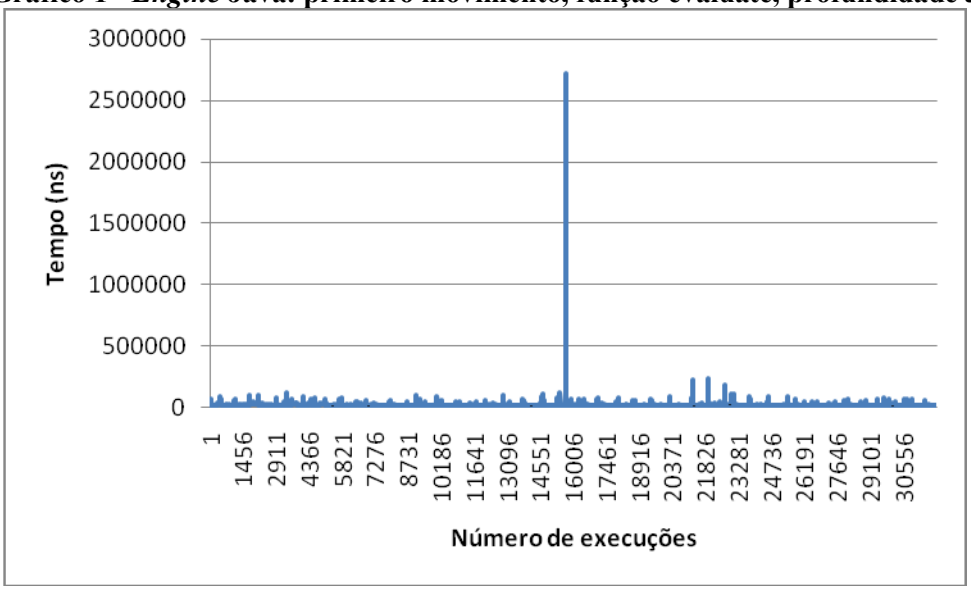
Fonte: Autoria própria

Na fase de final de jogo para as profundidades de busca 3 e 5 foram descartados os dois primeiros testes e realizados mais 10. Na Tabela 9 e Tabela 10 do Apêndice A apresentam-se os resultados.

Para obter uma correta interpretação dos resultados, foram realizadas medições de tempo das duas principais funções da *engine*. A função evaluate, de caráter não recursivo, que contém o conhecimento para o cálculo heurístico e a função recursiva zWSearch que realiza a busca na variação principal da árvore de busca. Os dados foram normatizados em nanosegundos e foram colocados em gráficos de linha. A visualização gráfica destes dados permite verificar se houve uma taxa elevada de interferências não inerentes aos algoritmos testados, tais como: tempo compartilhado, coleta de lixo, entre outras possibilidades.

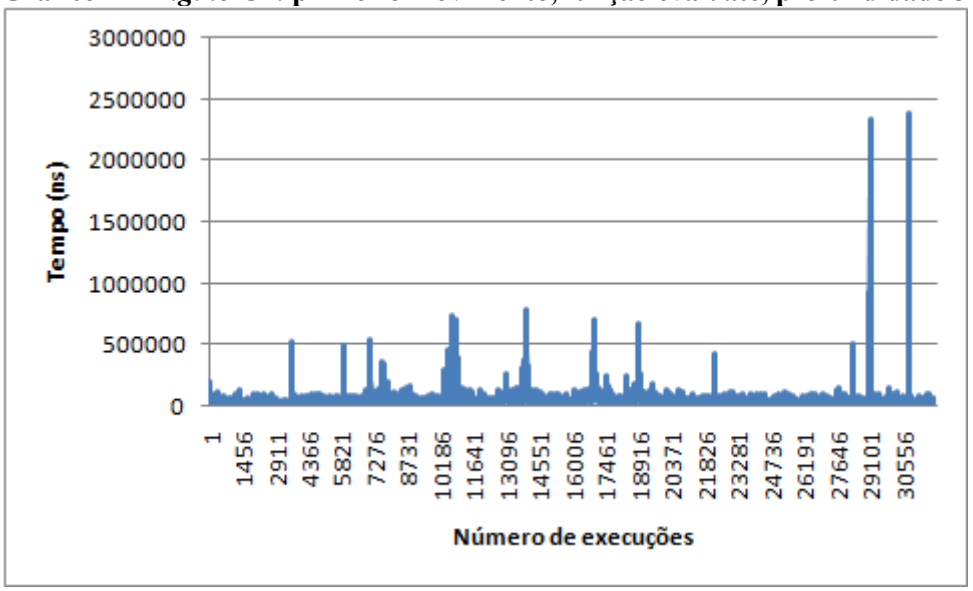
A ferramenta utilizada para geração dos gráficos de linha foi o Excel, que permitiu utilizar no máximo trinta e dois mil valores. Dessa forma foram utilizados os registros dos tempos de execução das trinta e duas mil primeiras chamadas dos métodos analisados. Os resultados podem ser visualizados a partir do Gráfico 1 até o Gráfico 32.

Gráfico 1 - *Engine* Java: primeiro movimento, função evaluate, profundidade 3



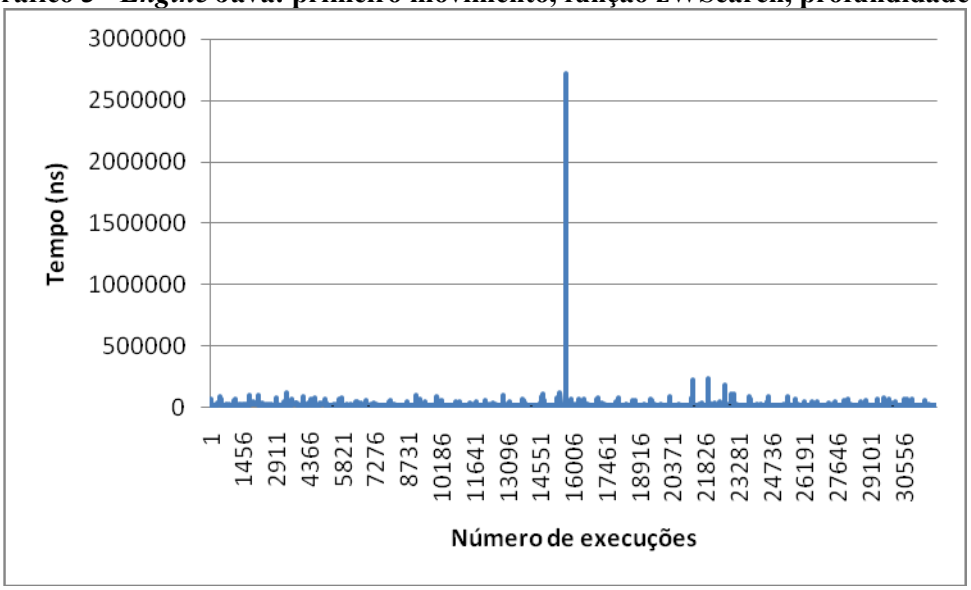
Fonte: Autoria própria.

Gráfico 2 - *Engine* C#: primeiro movimento, função evaluate, profundidade 3



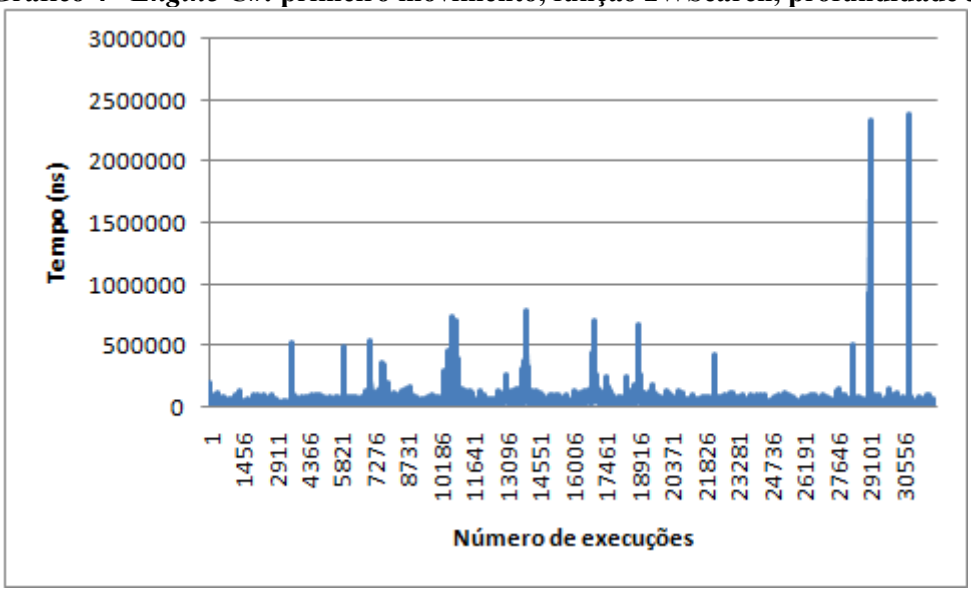
Fonte: Autoria própria.

Gráfico 3 - *Engine* Java: primeiro movimento, função zWSearch, profundidade 3



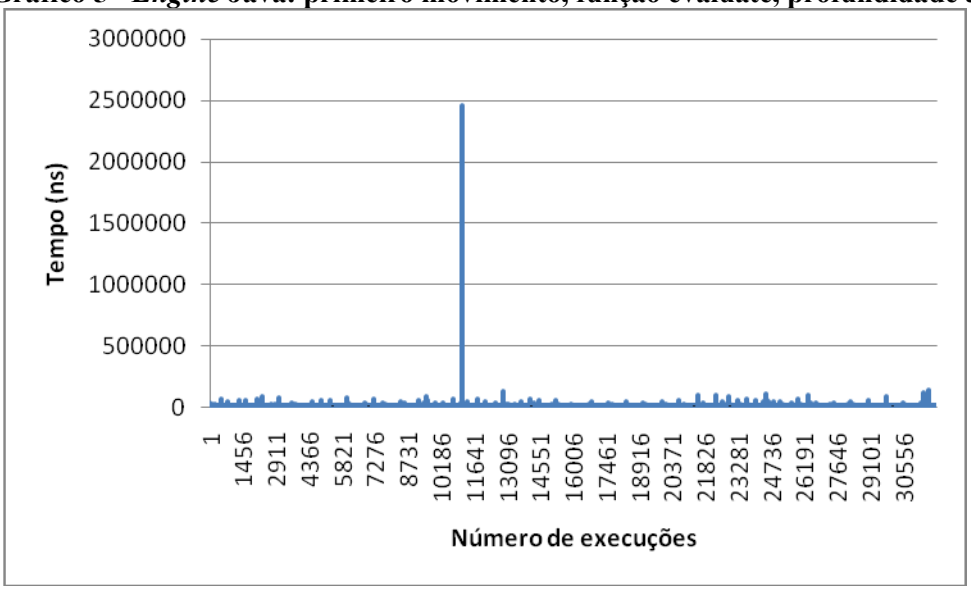
Fonte: Autoria própria.

Gráfico 4 - *Engine* C#: primeiro movimento, função zWSearch, profundidade 3



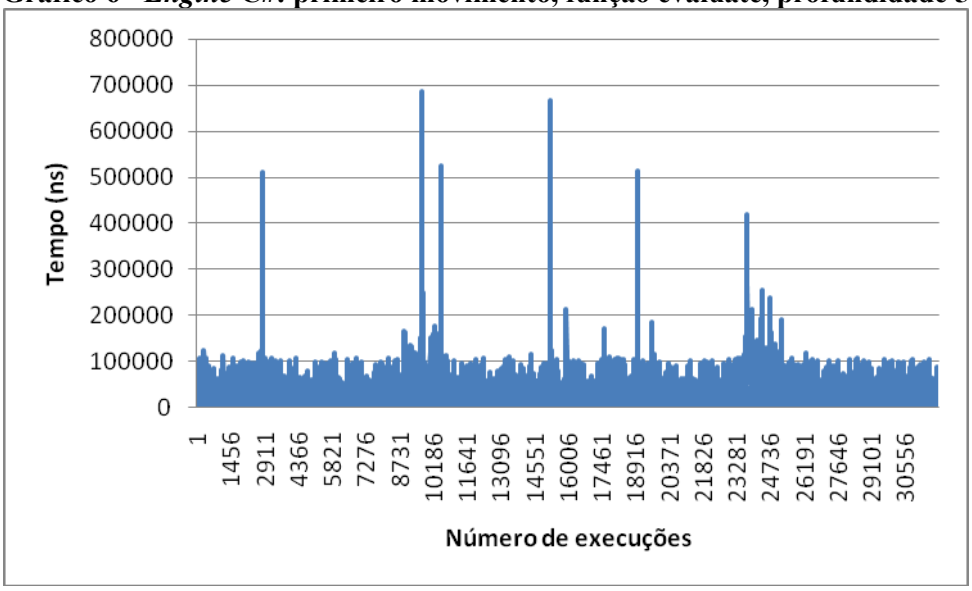
Fonte: Autoria própria.

Gráfico 5 - *Engine* Java: primeiro movimento, função evaluate, profundidade 5



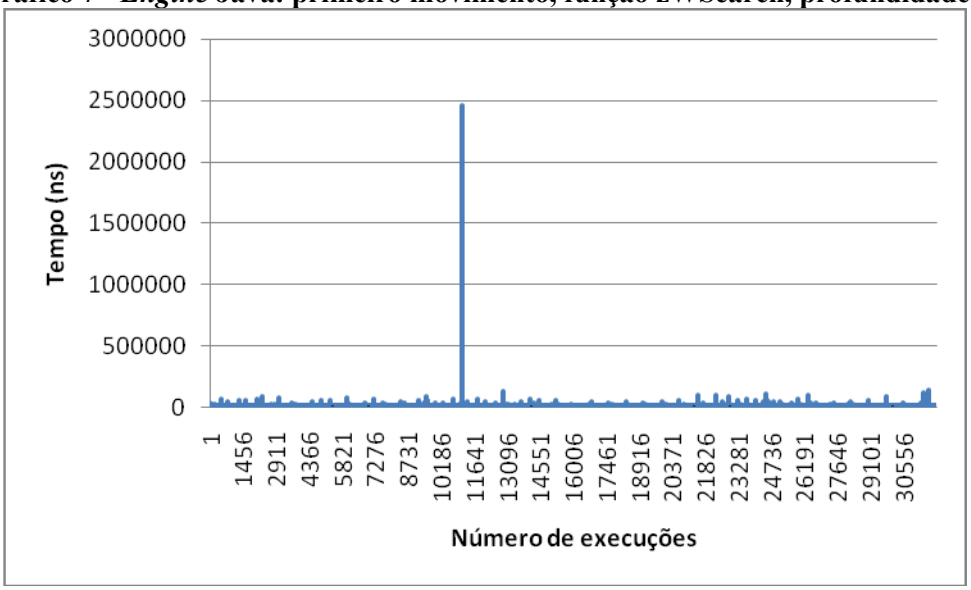
Fonte: Autoria própria.

Gráfico 6 - *Engine* C#: primeiro movimento, função evaluate, profundidade 5



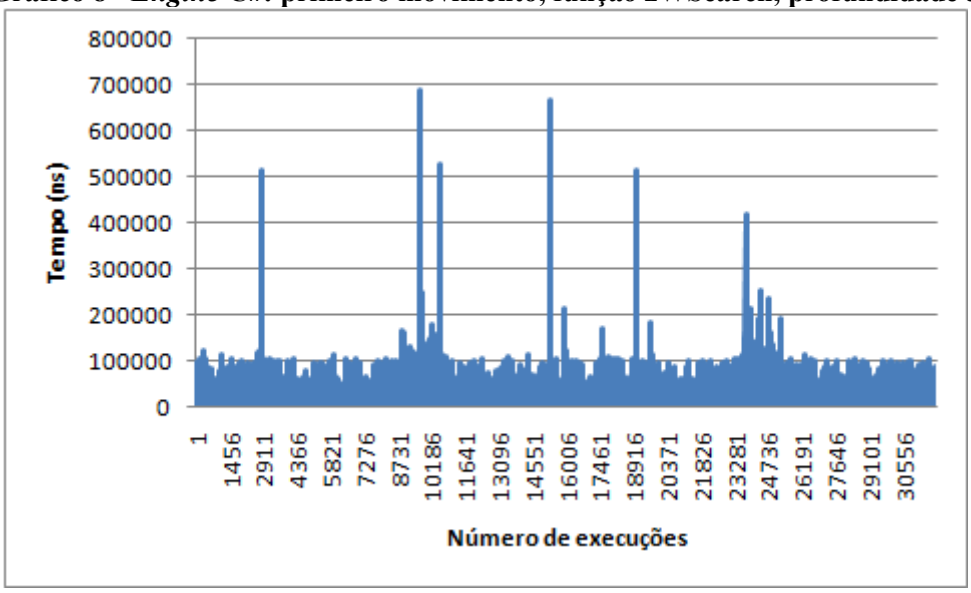
Fonte: Autoria própria.

Gráfico 7 - *Engine* Java: primeiro movimento, função zWSearch, profundidade 5



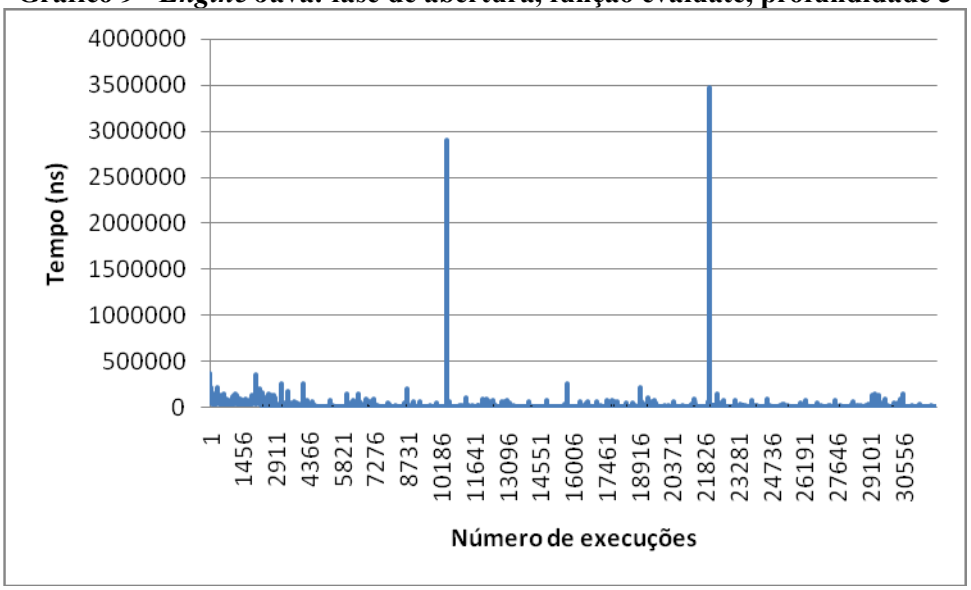
Fonte: Autoria própria.

Gráfico 8 - *Engine* C#: primeiro movimento, função zWSearch, profundidade 5



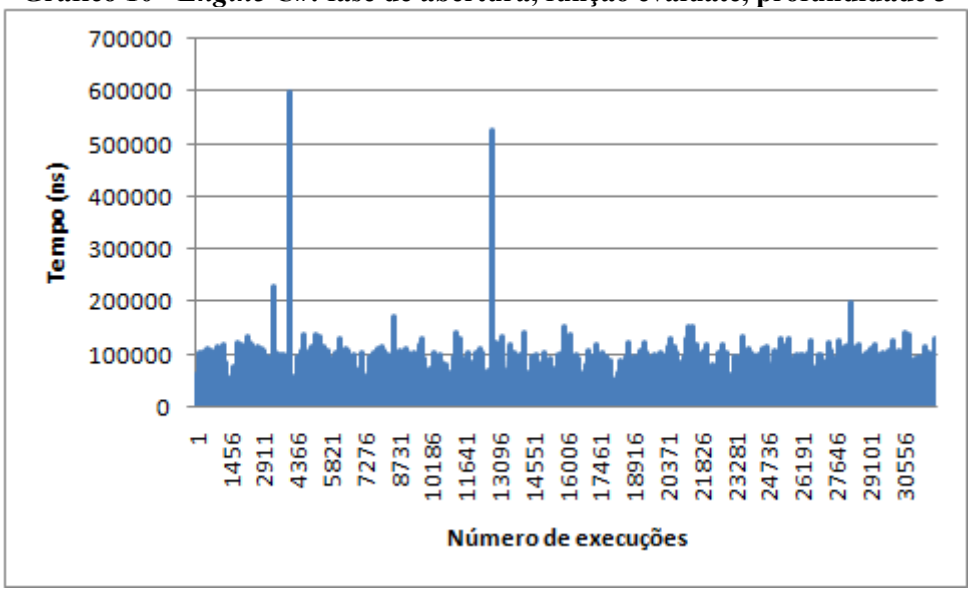
Fonte: Autoria própria.

Gráfico 9 - *Engine* Java: fase de abertura, função evaluate, profundidade 3



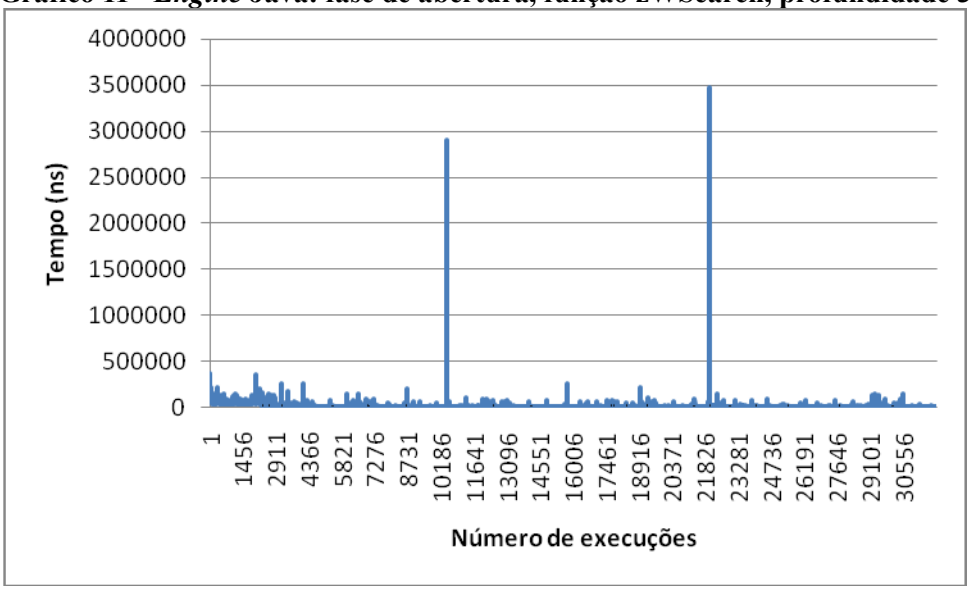
Fonte: Autoria própria.

Gráfico 10 - *Engine* C#: fase de abertura, função evaluate, profundidade 3



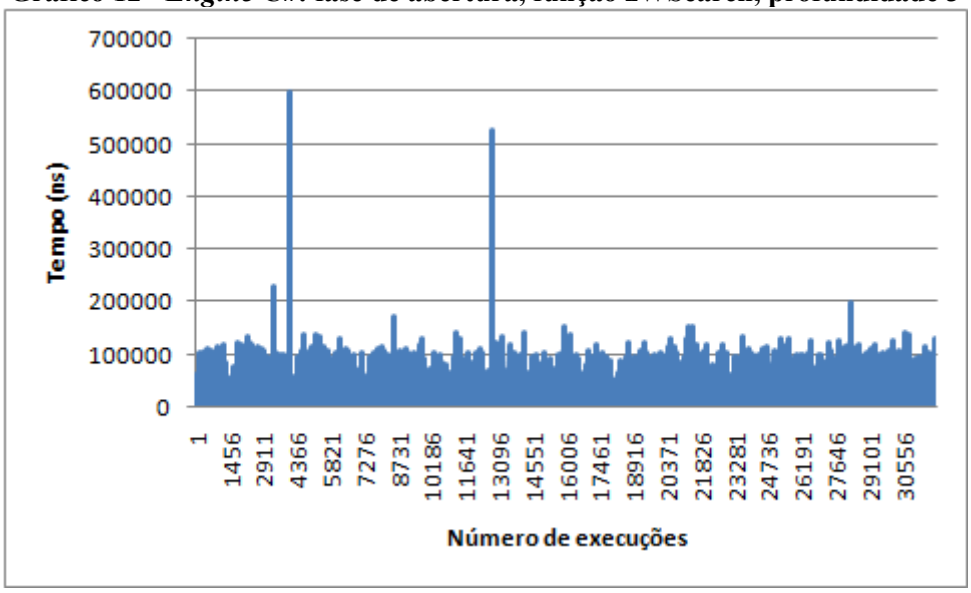
Fonte: Autoria própria.

Gráfico 11 - *Engine* Java: fase de abertura, função zWSearch, profundidade 3



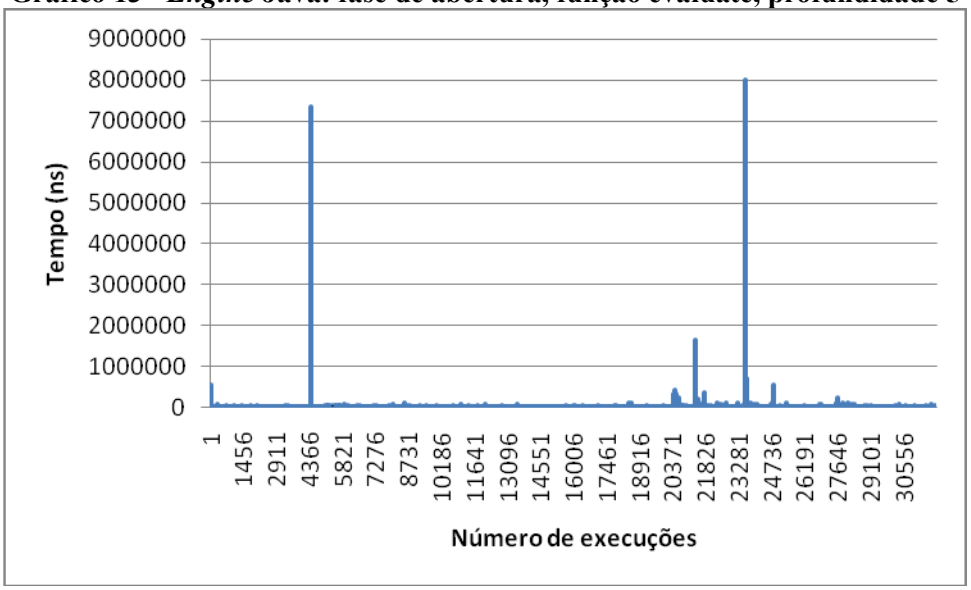
Fonte: Autoria própria.

Gráfico 12 - *Engine* C#: fase de abertura, função zWSearch, profundidade 3



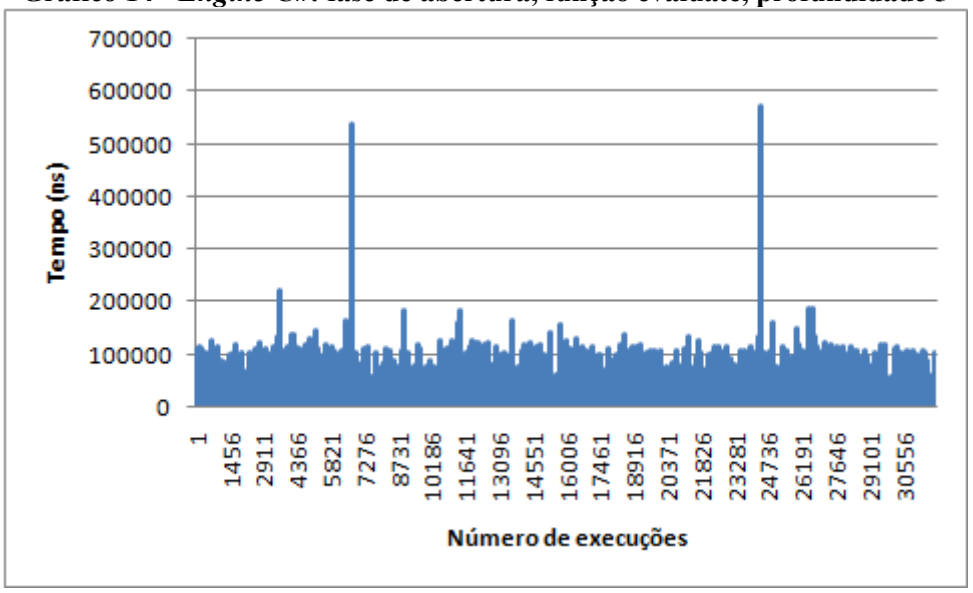
Fonte: Autoria própria.

Gráfico 13 - *Engine* Java: fase de abertura, função evaluate, profundidade 5



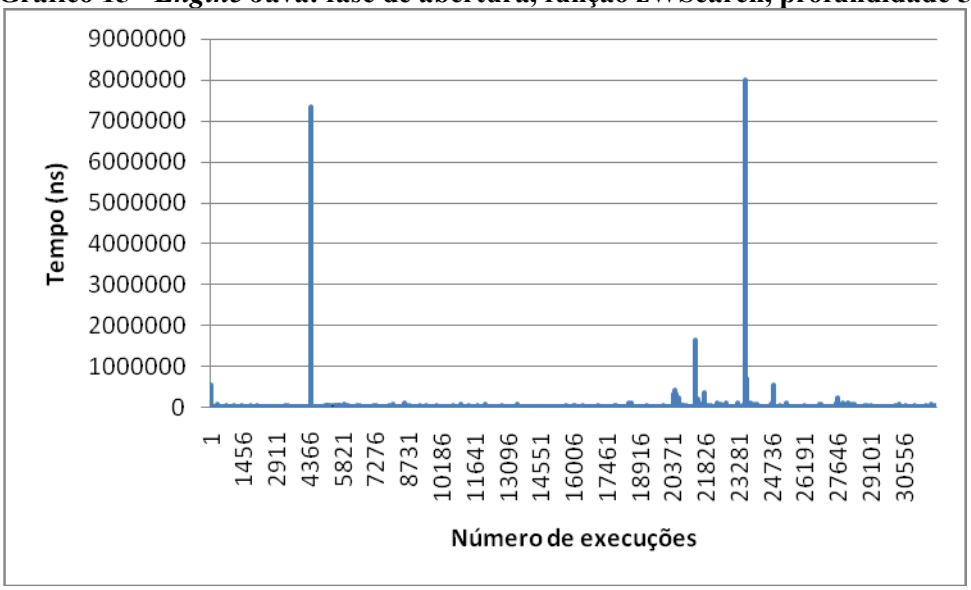
Fonte: Autoria própria.

Gráfico 14 - *Engine* C#: fase de abertura, função evaluate, profundidade 5



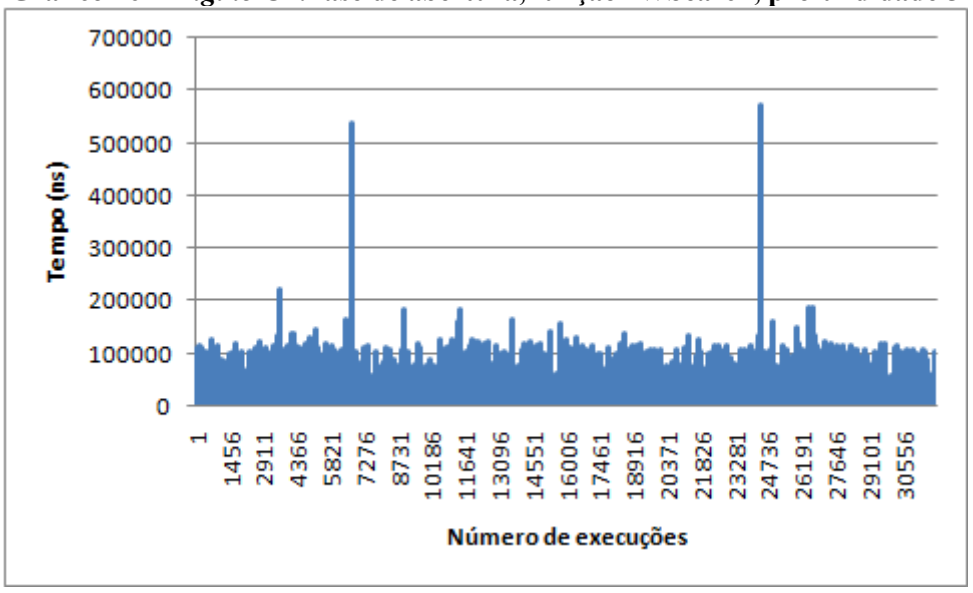
Fonte: Autoria própria.

Gráfico 15 - *Engine* Java: fase de abertura, função zWSearch, profundidade 5



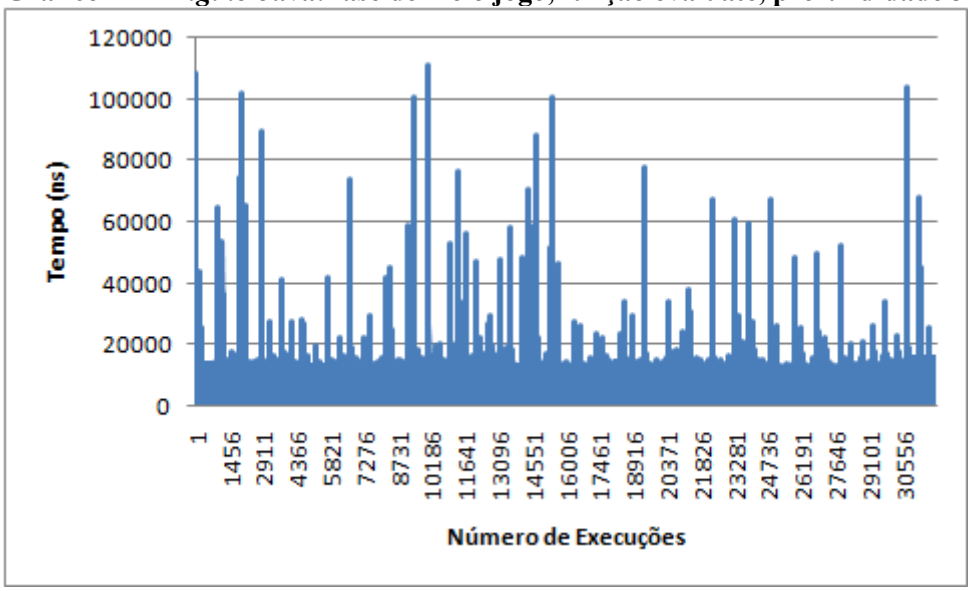
Fonte: Autoria própria.

Gráfico 16 - *Engine* C#: fase de abertura, função zWSearch, profundidade 5



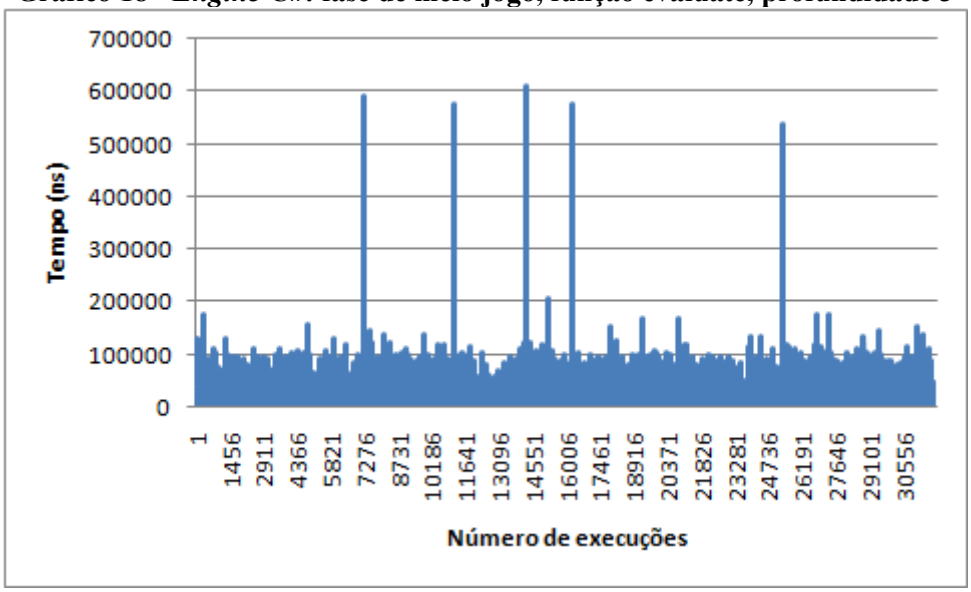
Fonte: Autoria própria.

Gráfico 17 - *Engine* Java: fase de meio jogo, função evaluate, profundidade 3



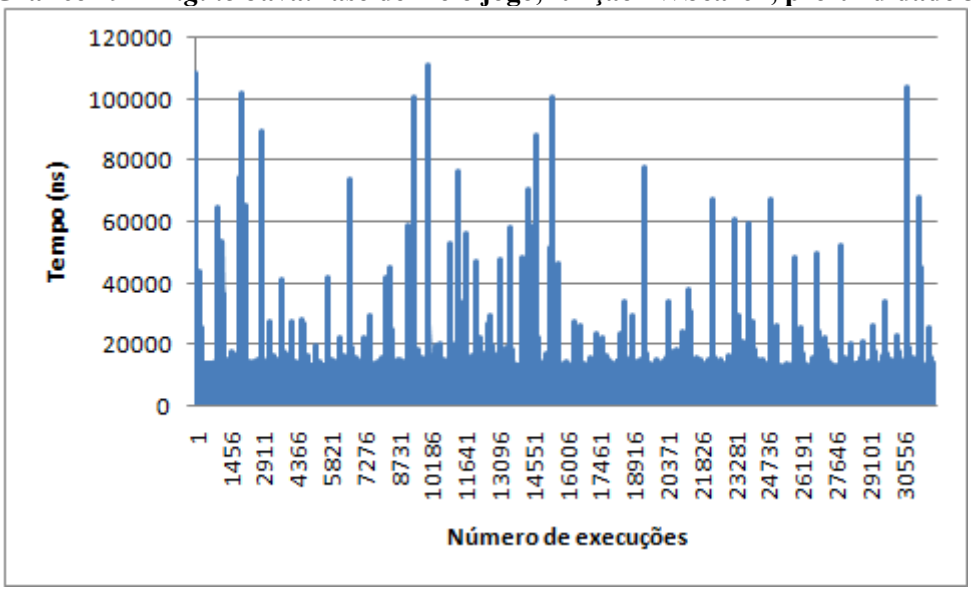
Fonte: Autoria própria.

Gráfico 18 - *Engine* C#: fase de meio jogo, função evaluate, profundidade 3



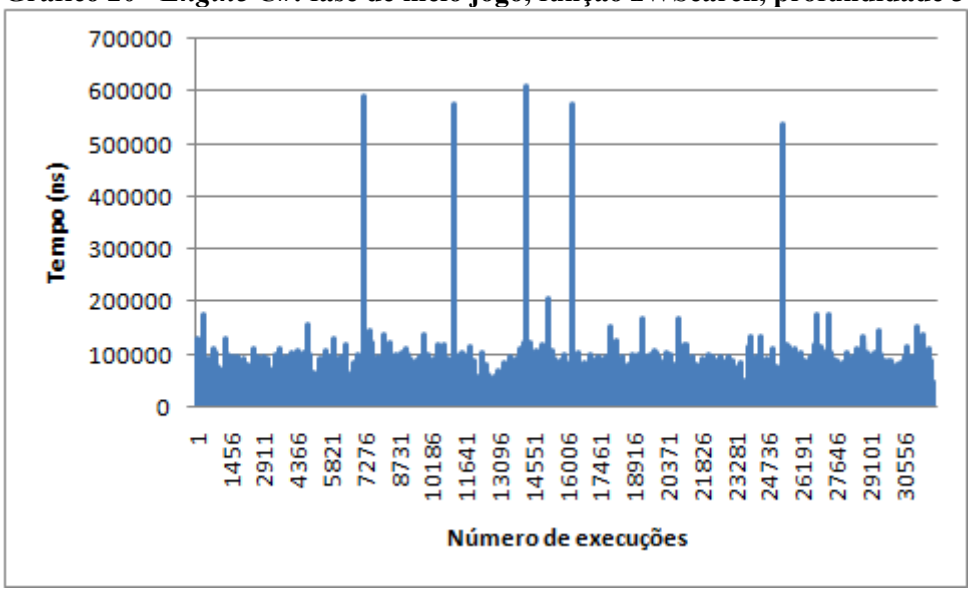
Fonte: Autoria própria.

Gráfico 19 - *Engine* Java: fase de meio jogo, função zWSearch, profundidade 3



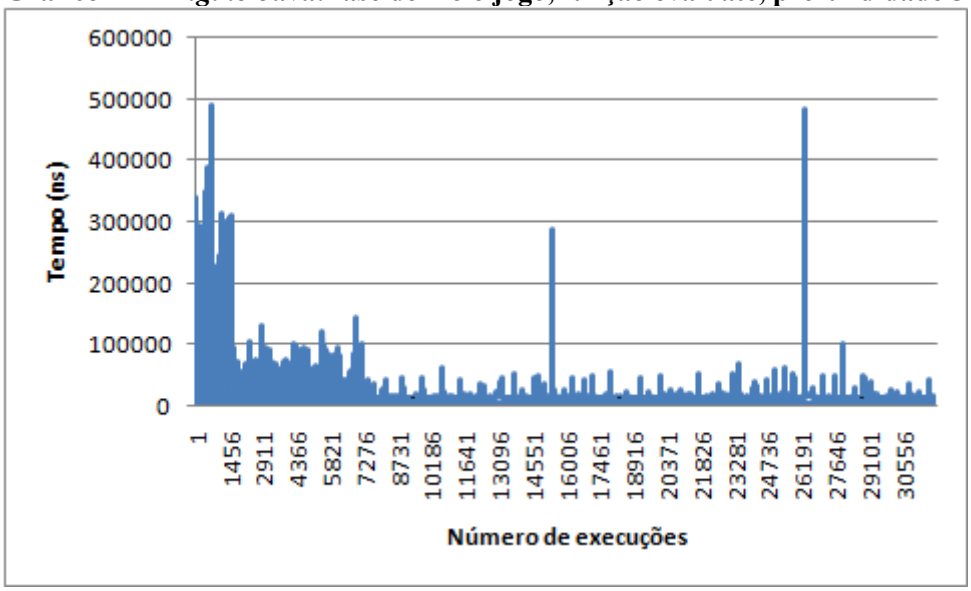
Fonte: Autoria própria.

Gráfico 20 - *Engine* C#: fase de meio jogo, função zWSearch, profundidade 3



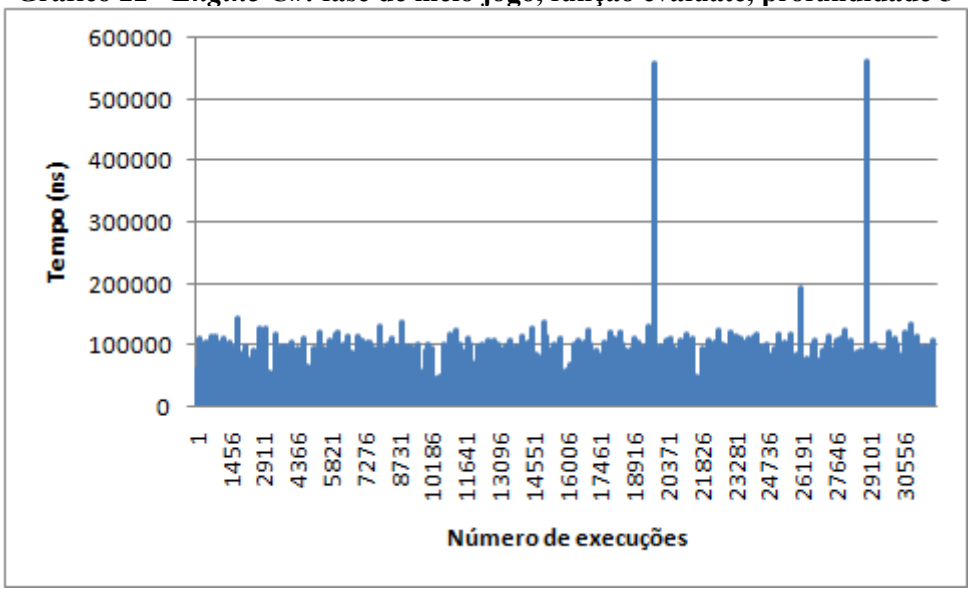
Fonte: Autoria própria.

Gráfico 21 - *Engine* Java: fase de meio jogo, função evaluate, profundidade 5



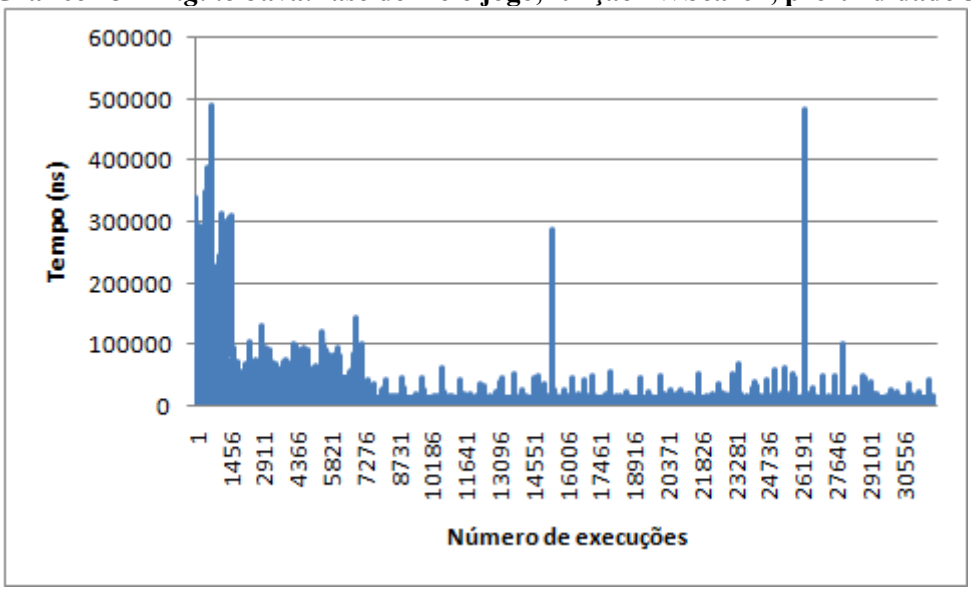
Fonte: Autoria própria.

Gráfico 22 - *Engine* C#: fase de meio jogo, função evaluate, profundidade 5



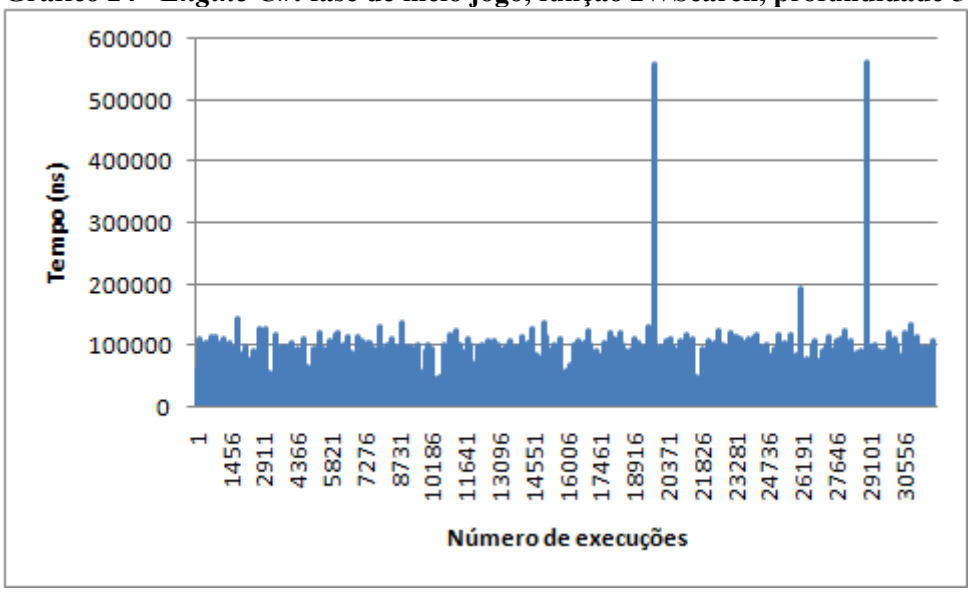
Fonte: Autoria própria.

Gráfico 23 - *Engine* Java: fase de meio jogo, função zWSearch, profundidade 5



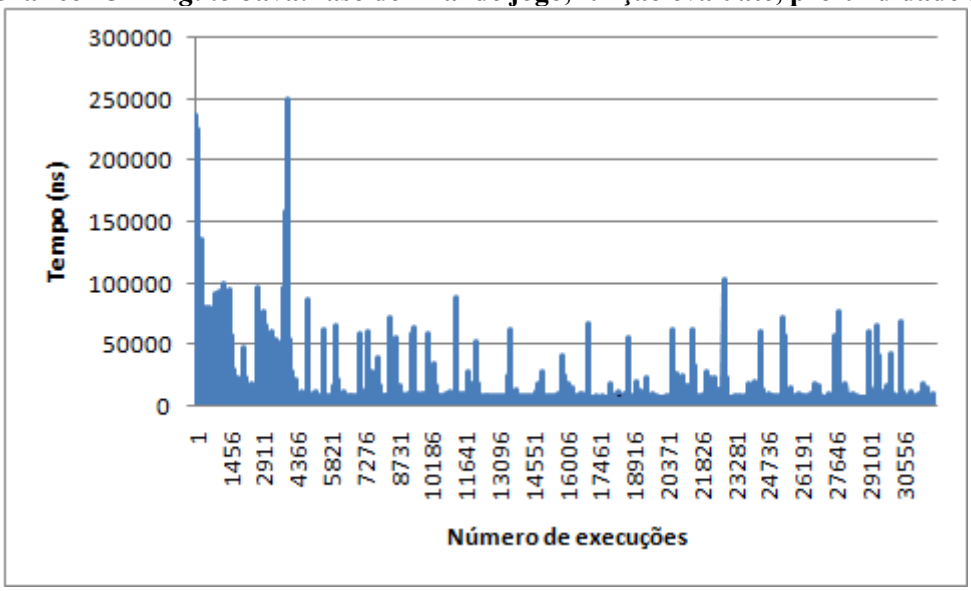
Fonte: Autoria própria.

Gráfico 24 - *Engine* C#: fase de meio jogo, função zWSearch, profundidade 5



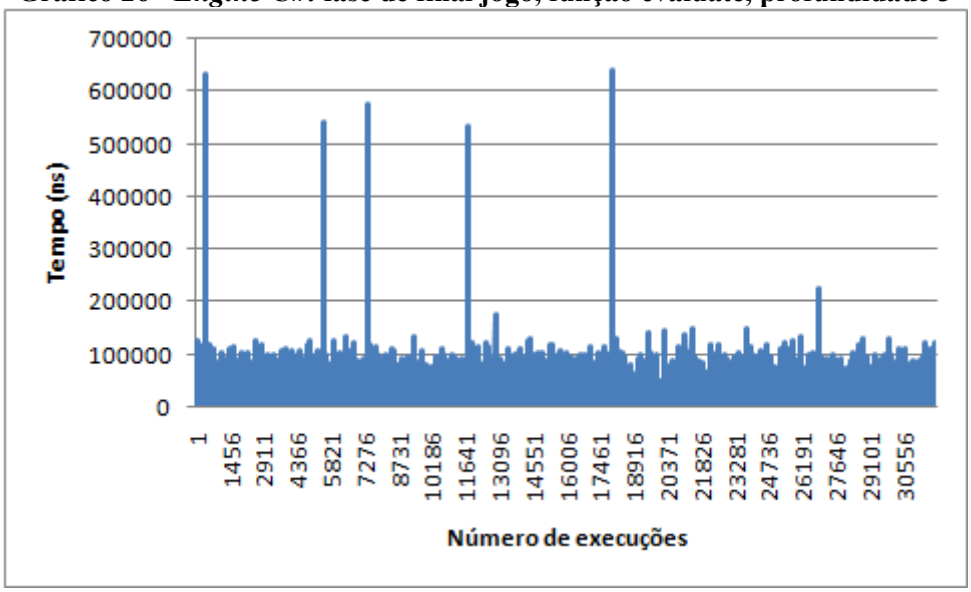
Fonte: Autoria própria.

Gráfico 25 - *Engine* Java: fase de final de jogo, função evaluate, profundidade 3



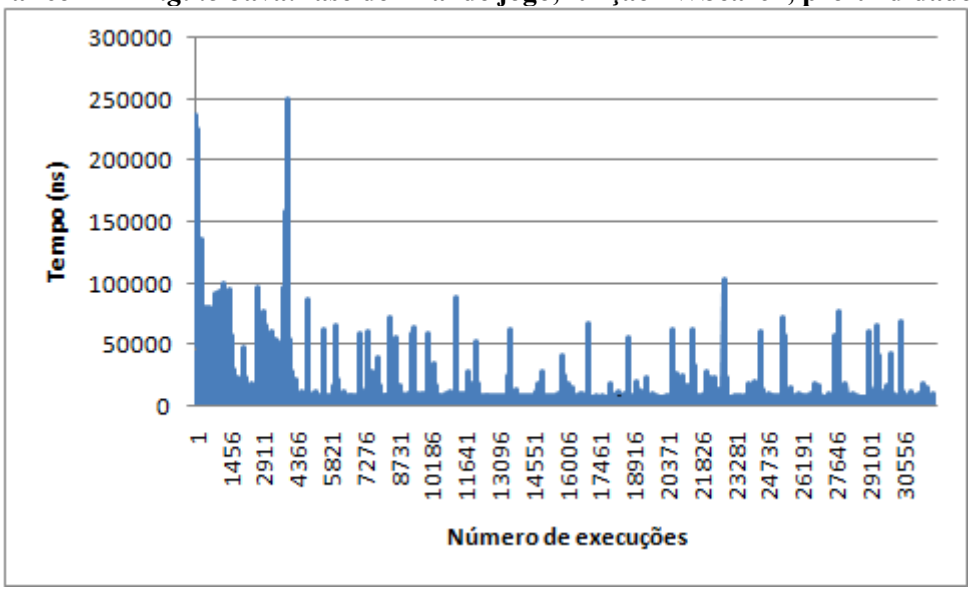
Fonte: Autoria própria.

Gráfico 26 - *Engine* C#: fase de final jogo, função evaluate, profundidade 3



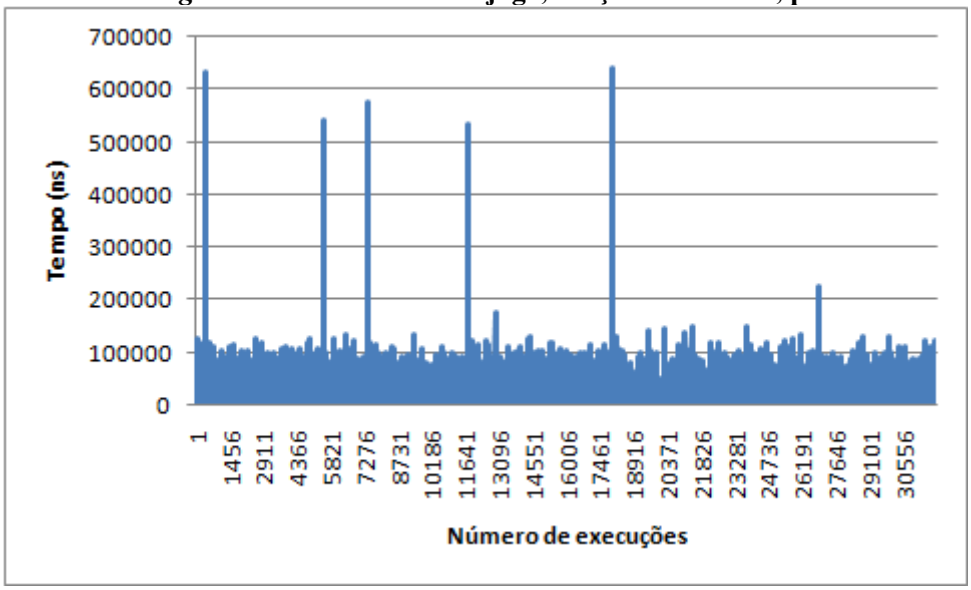
Fonte: Autoria própria.

Gráfico 27 - *Engine* Java: fase de final de jogo, função zWSearch, profundidade 3



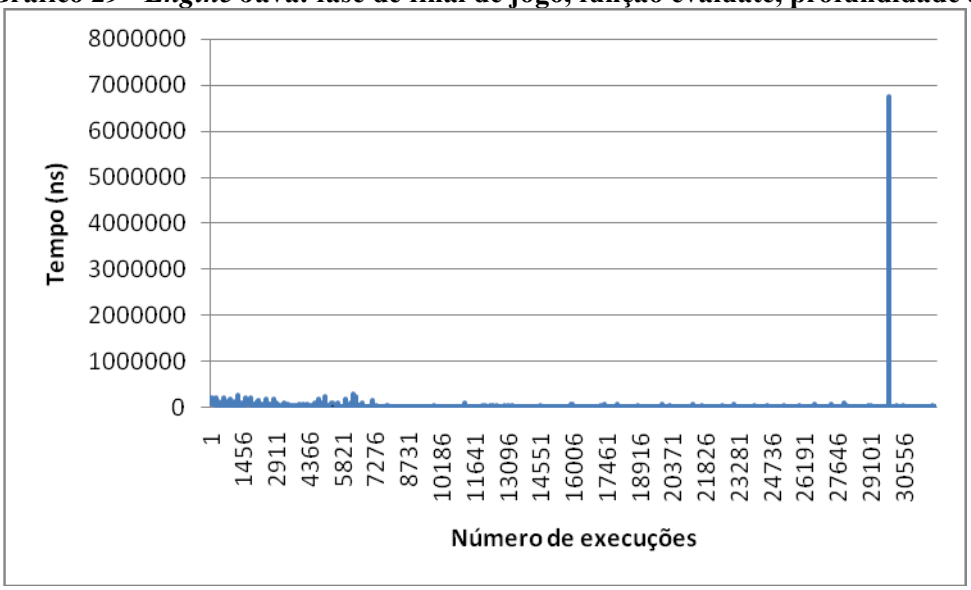
Fonte: Autoria própria.

Gráfico 28 - *Engine* C#: fase de final de jogo, função zWSearch, profundidade 3



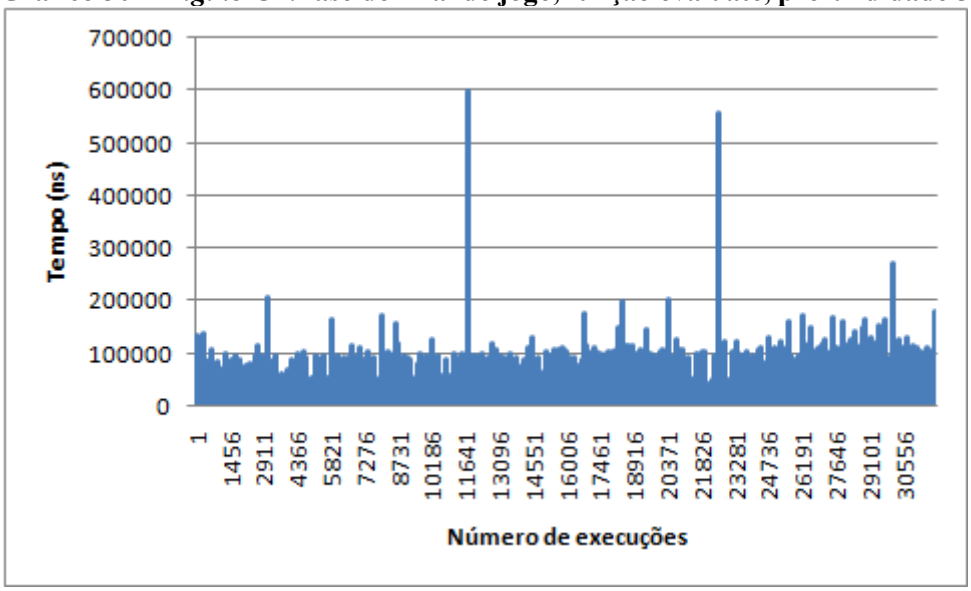
Fonte: Autoria própria.

Gráfico 29 - *Engine* Java: fase de final de jogo, função evaluate, profundidade 5



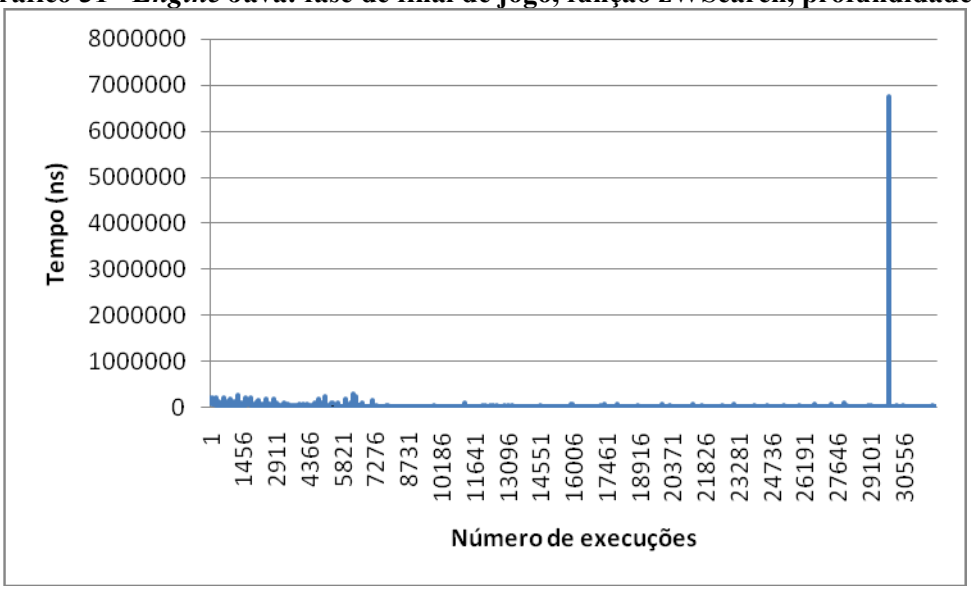
Fonte: Autoria própria.

Gráfico 30 - *Engine* C#: fase de final de jogo, função evaluate, profundidade 5



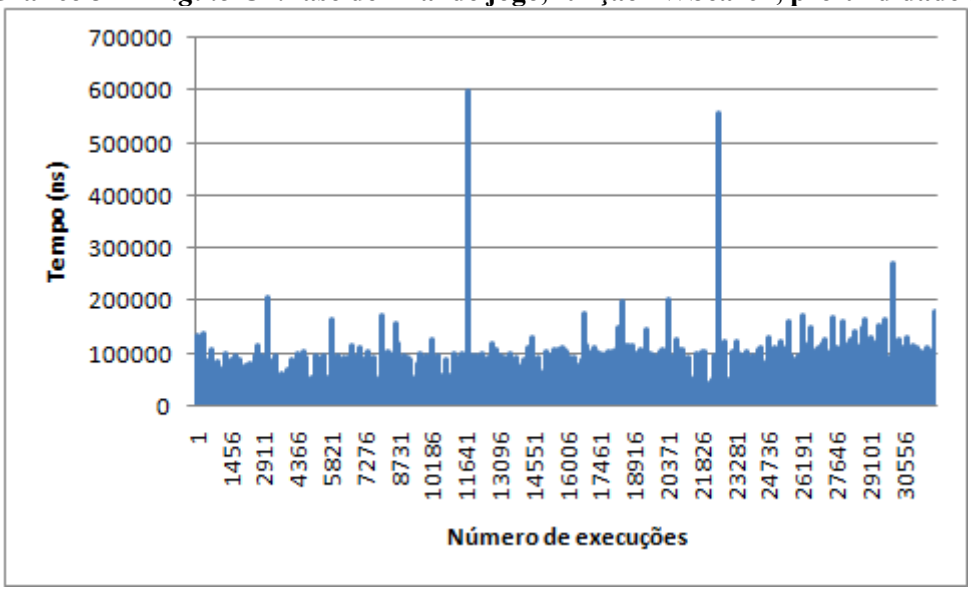
Fonte: Autoria própria.

Gráfico 31 - *Engine* Java: fase de final de jogo, função zWSearch, profundidade 5



Fonte: Autoria própria.

Gráfico 32 - *Engine* C#: fase de final de jogo, função zWSearch, profundidade 5



Fonte: Autoria própria.

5 DISCUSSÕES

Analisando os resultados, percebe-se que o código Java sempre apresentou, em pelo menos uma das chamadas realizadas, tempo de execução superior ou igual ao C#, para todos os métodos que foram calculados. Porém, mesmo assim no conjunto dos processos de buscas gerado através dos testes, o tempo total do Java ficou menor para praticamente todos os métodos, exceção do método rateMoveablitly que possuiu desempenho melhor para as fases de abertura e de meio de jogo.

De um modo geral a *engine* Java apresentou desempenho bastante superior, destacando-se principalmente os métodos recursivos, em que mesmo em seu pior tempo médio esteve 60 por cento mais rápido que o C#, como pode ser observado por meio do zWSearch, chegando ainda próximo a 100 por cento em todos os casos analisados para o método pvSearch. A diferença entre os tempos totais das chamadas medidos em milissegundos e o percentual de desempenho que a *engine* C# apresentou em relação à *engine* Java encontram-se descritos na Tabela 1.

Tabela 1 - Diferenças de tempo e percentual de desempenho da engine C# com relação à Java

Método	Fase de jogo	Profundidade	Diferença (ms)	Percentual
Flipboard	1º movimento	3	703	-83,590963
kingSafe	1º movimento	3	208	-36,048527
RateAttack	1º movimento	3	432	-44,307692
RateMaterial	1º movimento	3	116	-75,324675
RateMoveablitly	1º movimento	3	3	-33,333333
RatePositional	1º movimento	3	295	-85,260116
Evaluate	1º movimento	3	1669	-66,151407
GetFirstLegalMove	1º movimento	3	0	0
ZwSearch	1º movimento	3	2646	-72,255598
PvSearch	1º movimento	3	3638	-98,697775
Flipboard	1º movimento	5	18702	-82,8292
kingSafe	1º movimento	5	5602	-35,3372
RateAttack	1º movimento	5	11679	-43,5248
RateMaterial	1º movimento	5	3285	-76,8241
RateMoveablitly	1º movimento	5	72	-28,9157
RatePositional	1º movimento	5	8051	-85,4671
Evaluate	1º movimento	5	45107	-65,5758
GetFirstLegalMove	1º movimento	5	1	-100
ZwSearch	1º movimento	5	96525	-97,2094
PvSearch	1º movimento	5	97947	-98,6126
Flipboard	Abertura	3	7046	-79,3915
KingSafe	Abertura	3	1163	-19,4938
RateAttack	Abertura	3	3089	-30,5056
RateMaterial	Abertura	3	1160	-70,5167

Método	Fase de jogo	Profundidade	Diferença (ms)	Percentual
RateMoveablitly	Abertura	3	12	13,95349
RatePositional	Abertura	3	3224	-84,1336
Evaluate	Abertura	3	15406	-57,7761
GetFirstLegalMove	Abertura	3	1	-100
ZWSearch	Abertura	3	25734	-66,4
PvSearch	Abertura	3	38494	-99,1883
Flipboard	Abertura	5	131443	-79,97554
kingSafe	Abertura	5	21338	-18,84133
RateAttack	Abertura	5	56403	-29,5745
RateMaterial	Abertura	5	21360	-69,893
RateMoveablitly	Abertura	5	325	20,036991
RatePositional	Abertura	5	61374	-82,8796
Evaluate	Abertura	5	289748	-57,96003
GetFirstLegalMove	Abertura	5	15	-65,21739
ZWSearch	Abertura	5	495463	-66,86275
PvSearch	Abertura	5	725375	-97,76431
Flipboard	Meio de jogo	3	7022	-83,4066
kingSafe	Meio de jogo	3	952	-25,5433
RateAttack	Meio de jogo	3	2847	-40,1212
RateMaterial	Meio de jogo	3	1302	-75
RateMoveablitly	Meio de jogo	3	3	3,571429
RatePositional	Meio de jogo	3	3741	-85,7831
Evaluate	Meio de jogo	3	16121	-67,9122
GetFirstLegalMove	Meio de jogo	3	2	-66,6667
ZWSearch	Meio de jogo	3	26471	-74,1921
PvSearch	Meio de jogo	3	35498	-99,0541
Flipboard	Meio de jogo	5	364569	-79,3386
kingSafe	Meio de jogo	5	29634	-14,53
RateAttack	Meio de jogo	5	123741	-31,9152
RateMaterial	Meio de jogo	5	65986	-70,5997
RateMoveablitly	Meio de jogo	5	1123	24,45025
RatePositional	Meio de jogo	5	177366	-84,9238
Evaluate	Meio de jogo	5	770823	-61,1165
GetFirstLegalMove	Meio de jogo	5	13	-61,9048
ZWSearch	Meio de jogo	5	1316281	-69,266
PvSearch	Meio de jogo	5	1859572	-97,7961
Flipboard	Final de jogo	3	780	-83,1557
kingSafe	Final de jogo	3	35	-24,4755
RateAttack	Final de jogo	3	219	-54,0741
RateMaterial	Final de jogo	3	159	-80,303
RateMoveablitly	Final de jogo	3	1	-11,1111
RatePositional	Final de jogo	3	603	-86,2661
Evaluate	Final de jogo	3	1893	-76,3923
GetFirstLegalMove	Final de jogo	3	1	-100
ZWSearch	Final de jogo	3	2294	-76,9282
PvSearch	Final de jogo	3	3015	-99,4393
Flipboard	Final de jogo	5	23200	-84,0154
kingSafe	Final de jogo	5	843	-20,038
RateAttack	Final de jogo	5	6149	-51,8291

Método	Fase de jogo	Profundidade	Diferença (ms)	Percentual
RateMaterial	Final de jogo	5	4712	-81,1854
RateMoveablitly	Final de jogo	5	58	20,20906
RatePositional	Final de jogo	5	17686	-87,0374
Evaluate	Final de jogo	5	54690	-75,5272
GetFirstLegalMove	Final de jogo	5	8	-72,7273
ZWSearch	Final de jogo	5	68059	-76,8568
PvSearch	Final de jogo	5	88350	-99,2295

Fonte: Autoria própria.

O número de posições avaliadas pode ser obtido pela soma do número de chamadas dos métodos recursivos zWSearch e pvSearch. O número médio de posições avaliadas por segundo pelas *engines* é apresentado na Tabela 2, calculado para cada estado de jogo e profundidade correspondente.

Tabela 2 - Número médio de posições avaliadas por segundo pelas *engines*

Fase de jogo	Profundidade de busca	Posições avaliadas por segundo em Java	Posições avaliadas por segundo em C#
1º movimento	3	2819,54	408,27
1º movimento	5	949,62	19,83
Abertura	3	639,57	109,97
Abertura	5	644,15	113,86
Meio de jogo	3	2908,76	388,30
Meio de jogo	5	349,83	57,59
Final de jogo	3	11645,39	1365,14
Final de jogo	5	4311,14	514,16

Fonte: Autoria própria.

Analisando os gráficos obtidos fica evidente que a *engine* C# apresentou um volume maior de tempo na maioria das execuções. Apesar de a *engine* Java apresentar picos de tempo de execução bem maiores, na média geral a *engine* C# teve pior desempenho.

Também é possível afirmar que uma tradução direta de código de Java para C# degrada o desempenho. Assim, é preciso fazer importantes alterações no código para alcançar desempenho equivalente ou melhor.

6 CONCLUSÃO

Durante esse trabalho foi realizada a migração de uma *engine* de xadrez em código Java que implementa o algoritmo de busca MINIMAX com poda alfa beta para sua versão em C#, com a finalidade de identificar em qual linguagem a *engine* seria mais eficiente.

O primeiro passo foi o estudo das classes em ambas as linguagens para possibilitar a tradução do código, passando por adaptações de funções e criação de classes que permitissem comparação do desempenho entre as linguagens.

Após a tradução do código, para garantir que ambas apresentavam os mesmos resultados, as *engines* foram integradas com uma interface gráfica e foram realizados testes de validação, através da execução de sequências de movimentos por um jogador humano em diferentes estados de jogo e profundidades de busca. Dado que a tradução do código mostrou-se correta, devido às *engines* apresentarem os mesmos resultados para as buscas em situações de jogo idênticas, deu-se início aos testes de desempenho.

Durante os testes de desempenho foram analisados os principais métodos utilizados no processo de busca, incluindo métodos recursivos. Foram analisados os números totais de chamadas dessas funções durante os testes, permitindo identificar o maior número médio de posições avaliadas por segundo em cada *engine*, assim como seus tempos de execução, calculados para diferentes fases de jogo e profundidades de busca.

Realizando uma análise a partir de gráficos, foi possível identificar picos nos tempos de execução que podem ser devidos à interferência externa, não inerente ao algoritmo em si. Foram usados dois métodos, um linear e outro recursivo para diferentes estados de jogo e profundidade de busca em ambas as *engines,* como fonte de dados para os gráficos.

Quanto à migração, considera-se que foi bem sucedida, pois ambas as *engines* apresentaram a escolha dos mesmos movimentos em posições e profundidade de buscas iguais. Quanto à análise de desempenho comparativo, foi considerada como a mais eficiente, aquela que apresentou menores tempos médios para conclusão da execução de seus principais métodos utilizados no processo de busca.

6.1 LIMITAÇÕES

O presente trabalho apresenta algumas limitações, tanto do ponto de vista da análise de desempenho quanto da experiência de jogo com ambas as *engines*:

1. A máquina de testes não é dedicada, o que pode ter ocasionado algumas das interferências registradas nos gráficos, aumentando consideravelmente o tempo de resposta dos métodos.
2. A *engine* original em Java, não estava completa, faltando uma função heurística adequada para jogo, com mais conhecimento representado, bem como a tabela de chaves (*hash),* usando o método criado por Zobrist (ZOBRIST HASHING, 2020).
3. Devido ao baixo desempenho de ambas as *engines,* não foi possível atingir uma profundidade de busca maior, como as principais *engines* existentes atualmente, que conseguem atingir mais de 15 níveis, processando milhões de posições por segundo. Isto torna seu uso inadequado.
4. As *engines* deste trabalho não incorporam conhecimentos importantes do xadrez como os livros de aberturas e finais de jogo.
5. Não foi possível gerar o gráfico de todas as medições devido às limitações do Excel. Apenas as 32000 primeiras medições foram consideradas nos gráficos.

6.2 TRABALHOS FUTUROS

1. Implementar a *engine* em uma outra linguagem como C++, a fim de comparar desempenho.
2. Incorporar a tabela Zobrist e os livros de abertura (ZOBRIST HASHING, 2020).
3. Ampliar o conhecimento heurístico da função de avaliação.
4. Otimizar os algoritmos para alcançar profundidades maiores, gastando menos tempo.

REFERÊNCIAS

ESCOLANO RUIZ, F.; RIZO ALDEGUER, R.; COMPAÑ ROSIQUE, P.; CAZORLA QUEVEDO, MIGUEL ANGEL. **Fundamentos de Inteligência Artificial**. Universidade de Alicante, 1999.

SAKS, M.; WIGDERSON, A. (1986). **"Probabilistic Boolean Decision Trees and the Complexity of Evaluating Game Trees"**. 27th Annual Symposium on Foundations of Computer Science. p. 29–38. doi:10.1109/SFCS.1986.44. ISBN 0-8186-0740-8.

WARKENTIN, Jonathan. **Programming an Advanced Java Chess Engine – Logic Crazy**, 2014. Disponível em: https://www.youtube.com/playlist?list=PLQV5mozTHmacMeRzJCW_8K3qw2miYqd0c. Acesso em: 10 set. 2019.

D'LUCIA, R. S. *et al.* **O ensino de Xadrez como ferramenta no processo de aprendizado infantil**. Revista Ciência em Extensão, v.3, n.2, 2007.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial**. 3.ed. Rio de Janeiro: Elsevier, 2013.

EVALUATION. In: Chessprogramming wiki. Disponível em: https://www.chessprogramming.org/index.php?title=Evaluation&oldid=12338. Acesso em: 10 set. 2019.

SCARPAZZA, P. D. **What is the algorithm behind Stockfish, the chess engine?**, 2015. Disponível em: https://www.quora.com/What-is-the-algorithm-behind-Stockfish-the-chess-engine. Acesso em: 10 set. 2019.

FEITOSA, Alexandre *et al*. Definição Formal de Táticas de Xadrez por meio da Autoria Incremental de Conceitos Heurísticos. **Anais do XVIII Simpósio Brasileiro de Informática na Educação (SBIE)**, 2007, p. 244-253, São Paulo – SP.

BUENO, Luis *et al*. Visualização de táticas para apoiar a aquisição de habilidades em jogos educacionais. **Anais do XXVIII Congresso da SBC,** 2008, p. 380-389, Belém do Pará – PA.

NEUMANN VON, John; MORGENSTERN, Oskar. **The theory of games and economic behavior**. Princeton, 3.ed., 1990.

WARKENTIN, Jonathan. **Advanced Java Chess Engine**, 2014. Disponível em: https://onedrive.live.com/?authkey=%21ANA5AE6LntFmf3o&id=D4629BC8D856F7D5%21242&cid=D4629BC8D856F7D5. Acesso em: 10 set. 2019.

WARKENTIN, Jonathan. **Programming an Advanced Java Chess Engine – Logic Crazy**, 2014. Disponível em: https://www.youtube.com/playlist?list=PLQV5mozTHmacMeRzJCW_8K3qw2miYqd0c. Acesso em: 10 set. 2019.

ZOBRIST HASHING. In: Chessprogramming wiki. Disponível em: https://www.chessprogramming.org/Zobrist_Hashing. Acesso em: 09 nov. 2020.

APÊNDICE A

Tabela 3 - Resultados dos testes de desempenho: primeiro movimento com profundidade de busca 3

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	91840	138	0	3	0.0015026132404181184	841	0	1	0,00915722996515679
kingSafe	1469440	369	0	0	2.511160714285714E-4	577	0	2	0,000392666594076655
RateAttack	91840	543	0	0	0.0059124564459930315	975	0	2	0,0106162891986063
RateMaterial	91840	38	0	0	4.137630662020906E-4	154	0	0	0,00167682926829268
RateMoveablitly	91840	6	0	0	6.533101045296167E-5	9	0	0	9,79965156794425E-05
RatePositional	91840	51	0	0	5.553135888501742E-4	346	0	2	0,00376742160278746
Evaluate	45920	854	0	3	0.018597560975609755	2523	0	2	0,0549433797909408
GetFirstLegalMove	200	0	0	0	0.0	0	0	0	0
ZWSearch	2980	1016	-	-	0.3409395973154362	3662	-	-	1,22885906040268
PvSearch	20	48	-	-	2.4	3686	-	-	184,3

Fonte: Autoria própria

Tabela 4 - Resultados dos testes de desempenho: primeiro movimento com profundidade de busca 5

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	2649160	3877	0	10	0.0014634827643479442	22579	0	4	0,00852307901372511
kingSafe	42386560	10251	0	0	2.41845528393906E-4	15853	0	24	0,000374010063567319
RateAttack	2649160	15154	0	0	0.0057203037944103035	26833	0	24	0,0101288710383669
RateMaterial	2649160	991	0	0	3.7408084071932233E-4	4276	0	1	0,00161409654381011
RateMoveablitly	2649160	177	0	0	6.681363149073668E-5	249	0	0	9,39920578598499E-05
RatePositional	2649160	1369	0	8	5.167675791571668E-4	9420	0	19	0,00355584411662565
Evaluate	1324580	23679	0	10	0.017876609944284225	68786	0	24	0,0519304232284951
GetFirstLegalMove	240	0	0	0	0	1	0	0	0,00416666666666667
ZWSearch	3920	2771	-	-	7.069387755102041	99296	-	-	25,330612244898

PvSearch	20	1378	-	-	68.9	99325	-	-	4966,25

Fonte: Autoria própria

Tabela 5 - Resultados dos testes de desempenho: fase de abertura com profundidade de busca 3

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	1080080	1829	0	8	0.0016933930819939265	8875	0	0	0,00821698392711651
kingSafe	17281280	4803	0	4	2.779308014221169E-4	5966	0	0	0,000345229057106881
RateAttack	1080080	7037	0	4	0.006515258129027479	10126	0	0	0,00937523146433598
RateMaterial	1080080	485	0	0	4.4904081179171913E-4	1645	0	0	0,00152303533071624
RateMoveablitly	1080080	98	0	0	9.073401970224427E-5	86	0	0	7,96237315754389E-05
RatePositional	1080080	608	0	6	5.629212650914747E-4	3832	0	0	0,00354788534182653
Evaluate	540040	11259	0	8	0.020848455669950375	26665	0	1	0,0493759721502111
GetFirstLegalMove	350	0	0	0	0	1	0	0	0,00285714285714286
ZWSearch	8490	13022	-	-	1.5338044758539457	38756	-	-	4,56489988221437
PvSearch	40	315	-	-	7.875	38809	-	-	970,225

Fonte: Autoria própria

Tabela 6 - Resultados dos testes de desempenho: fase de abertura com profundidade de busca 5

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	20101100	32911	0	18	0.0016372735820427739	164354	0	8	0,0081763684574476
kingSafe	321617600	91913	0	13	2.857834894607758E-4	113251	0	3	0,000352129361079742
RateAttack	20101100	134312	0	13	0.006681823382799946	190715	0	3	0,00948778922546527
RateMaterial	20101100	9201	0	3	4.5773614379312577E-4	30561	0	2	0,00152036455716354
RateMoveablitly	20101100	1947	0	0	9.686037082547721E-5	1622	0	0	8,06921014272851E-05
RatePositional	20101100	12678	0	18	6.30711752093169E-4	74052	0	3	0,003683977493769
Evaluate	10050550	210162	0	18	0.02091049743546373	499910	0	8	0,049739566491386

GetFirstLegalMove	6850	8	0	0	0.0011678832116788322	23	0	0	0,00335766423357664
ZWSearch	168820	245552	-	-	1.4545196066816728	741015	-	-	4,38937922047151
PvSearch	40	16588	-	-	414.7	741963	-	-	18549,075

Fonte: Autoria própria.

Tabela 7 - Resultados dos testes de desempenho: fase de meio jogo com profundidade de busca 3

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	1043100	1397	0	9	0.001339277154635222	8419	0	1	0,00807113411945164
kingSafe	11474100	2775	0	3	2.4184903391115644E-4	3727	0	0	0,00032481850428356
RateAttack	1043100	4249	0	3	0.004073434953503979	7096	0	2	0,00680279934809702
RateMaterial	1043100	434	0	0	4.1606749113220207E-4	1736	0	0	0,00166426996452881
RateMoveablitly	1043100	87	0	0	8.340523439746908E-5	84	0	0	8,05291918320391E-05
RatePositional	1043100	620	0	8	5.943821301888601E-4	4361	0	3	0,00418080720928003
Evaluate	521550	7617	0	9	0.01460454414725338	23738	0	3	0,0455143322787844
GetFirstLegalMove	910	1	0	0	0.001098901098901099	3	0	0	0,0032967032967033
ZWSearch	27730	9208	0	20	0.3320591417237649	35679	0	71	1,28665705012622
PvSearch	40	339	0	117	8.475	35837	0	409	895,925

Fonte: Autoria própria.

Tabela 8 - Resultados dos testes de desempenho: fase de meio jogo com profundidade de busca 5

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	55646696	94941	0	39	0.0017061390311475097	459510	0	17	0,00825763312165021
kingSafe	612113656	174317	0	18	2.847788123844765E-4	203951	0	3	0,000333191390195026
RateAttack	55646696	263977	0	18	0.00474380365727374	387718	0	5	0,00696749363160753
RateMaterial	55646696	27479	0	14	4.938118877713781E-4	93465	0	3	0,00167961454530921
RateMoveablitly	55646696	5716	0	1	1.0271948580738738E-4	4593	0	0	8,25385931268947E-05
RatePositional	55646696	31487	0	22	5.658377273648017E-4	208853	0	3	0,00375319677560012

Evaluate	27823348	490413	0	39	0.017625952132000793	1261236	0	17	0,0453301306514227
GetFirstLegalMove	7152	8	0	0	0.0011185682326621924	21	0	0	0,00293624161073826
ZWSearch	218964	584048	-	-	2.6673243090188343	1900329	-	-	8,67872801008385
PvSearch	16	41907	-	-	2619.1875	1901479	-	-	118842,4375

Fonte: Autoria própria

Tabela 9 - Resultados dos testes de desempenho: fase de final de jogo com profundidade de busca 3

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	119340	158	0	5	0.0013239483827719122	938	0	0	0,00785989609519021
kingSafe	417690	108	0	0	2.585649644473174E-4	143	0	0	0,000342359165888578
RateAttack	119340	186	0	0	0.001558572146807441	405	0	0	0,00339366515837104
RateMaterial	119340	39	0	0	3.26797385620915E-4	198	0	0	0,00165912518853695
RateMoveablitly	119340	8	0	0	6.703536115300821E-5	9	0	0	7,54147812971342E-05
RatePositional	119340	96	0	0	8.044243338360985E-4	699	0	0	0,00585721468074409
Evaluate	59670	585	0	5	0.00980392156862745	2478	0	0	0,0415284062342886
GetFirstLegalMove	440	0	0	0	0	1	0	0	0,00227272727272727
ZWSearch	8170	688	-	-	0.08421052631578947	2982	-	-	0,36499388004896
PvSearch	40	17	-	-	0.425	3032	-	-	75,8

Fonte: Autoria própria

Tabela 10 - Resultados dos testes de desempenho: fase de final de jogo com profundidade de busca 5

Método	Chamadas	*Engine* Java				*Engine* CSharp			
		Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio	Tempo Total	Menor Tempo	Maior Tempo	Tempo Médio
Flipboard	3482220	4414	0	13	0.0012675821745897731	27614	0	1	0,00792999867900362
kingSafe	12187770	3364	0	0	2.7601439804000237E-4	4207	0	0	0,000345182096478683
RateAttack	3482220	5715	0	0	0.0016411944104622912	11864	0	0	0,00340702195725715
RateMaterial	3482220	1092	0	0	3.135930527077554E-4	5804	0	0	0,00166675281860422
RateMoveablitly	3482220	345	0	0	9.9074728190637E-5	287	0	0	8,24186869296024E-05

RatePositional	3482220	2634	0	12	7.564140117511242E-4	20320	0	3	0,00583535790386593
Evaluate	1741110	17721	0	13	0.010177989902992918	72411	0	3	0,0415889863363027
GetFirstLegalMove	4260	3	0	0	7.042253521126761E-4	11	0	0	0,00258215962441315
ZWSearch	91270	20494	-	-	0.22454256601292868	88553	-	-	0,97023118220664
PvSearch	40	686	-	-	17.15	89036	-	-	2225,9

Fonte: Autoria própria.

APÊNDICE B

Relatório com dados técnicos do equipamento utilizado para testes.

```
--------[ EVEREST Ultimate Edition ]-----------------------------------------------------------

    Versão                                          EVEREST v4.60.1500/pt
    Módulo de Benchmark                             2.3.237.0
    Homepage                                        http://www.lavalys.com/
    Tipo de relatório                               Assistente de relatórios [TRIAL VERSION]
    Computador                                      DESKTOP-91O94EG
    Gerador                                         asus
    Sistema operacional                             Windows 10 Home Home Edition 6.2.9200
    Data                                            2020-11-09
    Hora                                            20:04


--------[ Sumário ]----------------------------------------------------------------------------

    Computador:
      Tipo de Computador                             PC ACPI de base x64
      Sistema operacional                            Windows 10 Home Home Edition
      Service Pack do Sistema Operacional            [ TRIAL VERSION ]
      Internet Explorer                              9.11.19041.0
      DirectX                                        DirectX 12.0
      Nome do Computador                             DESKTOP-91O94EG
      Nome do usuário                                asus
      Nome do domínio                                [ TRIAL VERSION ]
      Data / Hora                                    2020-11-09 / 20:04

    Placa mãe:
      Tipo de processador                            2x , 2700 MHz
      Nome da Placa Mãe                              ASUSTeK COMPUTER INC. X556URK
      Chipset da Placa Mãe                           Desconhecido
      Memória do Sistema                             [ TRIAL VERSION ]
      Tipo de BIOS                                   Unknown (ü)

    Monitor:
      Adaptador gráfico                              Intel(R) HD Graphics 620  (1024 MB)
      Adaptador gráfico                              Intel(R) HD Graphics 620  (1024 MB)
      Adaptador gráfico                              Intel(R) HD Graphics 620  (1024 MB)
      Monitor                                        Monitor Genérico PnP [NoDB]

    Armazenamento:
      Controladora IDE                               Intel(R) 6th Generation Core Processor
Family Platform I/O SATA AHCI Controller
      Controladora "Storage"                        Controlador de Espaços de Armazenamento
da Microsoft
      Disco rígido                                   TOSHIBA MQ01ABD100  (931 GB)
      Status dos discos rígidos SMART                OK

    Partições:
      C: (NTFS)                                      [ TRIAL VERSION ]
      Tamanho total                                  [ TRIAL VERSION ]

    Dispositivos de entrada:
      Teclado                                         PC/AT Enhanced PS/2 Keyboard (101/102
teclas)
      Mouse                                          ASUS Touchpad

    DMI:
      Distribuidor da BIOS - DMI
      Versão da BIOS - DMI
      Fabricante do Sistema - DMI                    ASUSTeK COMPUTER INC.
      Nome do Sistema - DMI                          X556URK
      Versão do sistema - DMI                        1.0
      Número de série do Sistema - DMI               [ TRIAL VERSION ]
      UUID do Sistema - DMI                          [ TRIAL VERSION ]
      Fabricante da Placa Mãe - DMI                  ASUSTeK COMPUTER INC.
      Nome da Placa Mãe - DMI                        X556URK
      Versão da Placa Mãe - DMI                      1.0
      Número de série da Placa Mãe - DMI             [ TRIAL VERSION ]
```

```
      Fabricante do chassis - DMI                       ASUSTeK COMPUTER INC.
      Versão do chassis - DMI                           1.0
      Número de série do chassis - DMI                  [ TRIAL VERSION ]
      Identificador do chassis - DMI                    [ TRIAL VERSION ]
      Tipo de chassis - DMI                             Notebook


--------[ DMI ]---------------------------------------------------------------------------

  [ BIOS ]

    Propriedades da BIOS:
      Tamanho                                           64 KB
      Dispositivos de boot                              Floppy Disk, Hard Disk
      Funções disponíveis                               BBS, Smart Battery
      Padrões suportados                                DMI, ACPI
      Possibilidades de expansão                        USB

  [ Sistema ]

    Propriedades do Sistema:
      Fabricante                                        ASUSTeK COMPUTER INC.
      Produto                                           X556URK
      Versão                                            1.0
      Número de série                                   [ TRIAL VERSION ]
      Família                                           VivoBook
      ID Universal Único                                [ TRIAL VERSION ]
      Tipo de Wake-Up (acordar)                         Botão liga/desliga

  [ Placa mãe ]

    Propriedades da Placa Mãe:
      Fabricante                                        ASUSTeK COMPUTER INC.
      Produto                                           X556URK
      Versão                                            1.0
      Número de série                                   [ TRIAL VERSION ]

  [ Chassis ]

    Propriedades do chassis:
      Fabricante                                        ASUSTeK COMPUTER INC.
      Versão                                            1.0
      Número de série                                   [ TRIAL VERSION ]
      Etiqueta                                          [ TRIAL VERSION ]
      Tipo de chassis                                   Notebook
      Estado do Boot-Up                                 Seguro
      Estado da alimentação                             Seguro
      Condições de temperatura                          Seguro
      Condições de segurança                            Nenhum

  [ Processadores / Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz ]

    Propriedades do procesador:
      Fabricante                                        Intel(R) Corporation
      Versão                                                  Intel(R) Core(TM) i5-7200U CPU @
2.50GHz
      Número de série                                   To Be Filled By O.E.M.
      Etiqueta                                          To Be Filled By O.E.M.
      Número do tipo de componentes                     To Be Filled By O.E.M.
      Clock externo                                     100 MHz
      Velocidade de clock máxima                        8300 MHz
      Velocidade de clock atual                         2400 MHz
      Tipo                                              Central Processor
      Voltagem                                          0.8 V
      Estado                                            Ativado
      Identificação do socket                           U3E1
      Unidades HTT / CMP                                2 / 2

  [ Caches / L1 Cache ]

    Propriedades do cache:
      Tipo                                              Interna
      Estado                                            Ativado
      Modo de operação                                  Write-Back
      Associatividade                                   8-way Set-Associative
      Tamanho máximo                                    128 KB
      Tamanho instalado                                 128 KB
```

```
    Tipo de SRAM suportada                          Synchronous
    Tipo de SRAM atual                              Synchronous
    Correção de erros                               Parity
    Identificação do socket                         Cache L1

[ Caches / L2 Cache ]

  Propriedades do cache:
    Tipo                                            Interna
    Estado                                          Ativado
    Modo de operação                                Write-Back
    Associatividade                                 4-way Set-Associative
    Tamanho máximo                                  512 KB
    Tamanho instalado                               512 KB
    Tipo de SRAM suportada                          Synchronous
    Tipo de SRAM atual                              Synchronous
    Correção de erros                               Single-bit ECC
    Identificação do socket                         Cache L2

[ Caches / L3 Cache ]

  Propriedades do cache:
    Tipo                                            Interna
    Estado                                          Ativado
    Modo de operação                                Write-Back
    Tamanho máximo                                  3072 KB
    Tamanho instalado                               3072 KB
    Tipo de SRAM suportada                          Synchronous
    Tipo de SRAM atual                              Synchronous
    Correção de erros                               Multi-bit ECC
    Identificação do socket                         Cache L3

[ Periféricos de memória / ChannelA-DIMM0 ]

  Propriedades do dispositivo de memória:
    Forma                                           SODIMM
    Tipo detalhado                                  Synchronous
    Tamanho                                         4096 MB
    Velocidade                                      2133 MHz
    Tamanho total                                   64 bits
    Comprimento de dados                            64 bits
    Localização do dispositivo                      ChannelA-DIMM0
    Número do Banco                                 BANK 0
    Fabricante                                      0000
    Número de série                                 00000000
    Etiqueta                                        9876543210

[ Periféricos de memória / ChannelA-DIMM1 ]

  Propriedades do dispositivo de memória:
    Localização do dispositivo                      ChannelA-DIMM1
    Número do Banco                                 BANK 1

[ Periféricos de memória / ChannelB-DIMM0 ]

  Propriedades do dispositivo de memória:
    Forma                                           SODIMM
    Tipo detalhado                                  Synchronous
    Tamanho                                         4096 MB
    Velocidade                                      2400 MHz
    Tamanho total                                   64 bits
    Comprimento de dados                            64 bits
    Localização do dispositivo                      ChannelB-DIMM0
    Número do Banco                                 BANK 2
    Fabricante                                      Samsung
    Número de série                                 1879C5CA
    Etiqueta                                        9876543210
    Número do tipo de componentes                   M471A5244CB0-CRC

[ Periféricos de memória / ChannelB-DIMM1 ]

  Propriedades do dispositivo de memória:
    Localização do dispositivo                      ChannelB-DIMM1
    Número do Banco                                 BANK 3

[ Dispositivos integrados / VGA ]
```

```
    Propriedades do dispositivo integrado:
      Descrição                                     VGA
      Tipo                                          Video
      Estado                                        Ativado

  [ Dispositivos integrados / SATA Controller ]

    Propriedades do dispositivo integrado:
      Descrição                                     SATA Controller
      Tipo                                          SATA Controller
      Estado                                        Ativado


--------[ Overclock ]---------------------------------------------------------------------

    Propriedades da CPU:
      Tipo de processador                           2x
      (CPUID) Nome da CPU                                 Intel(R) Core(TM) i5-7200U CPU @
2.50GHz
      (CPUID) Revisão                               000806E9h

    Velocidade da CPU:
      Velocidade de clock do processador             2712.0 MHz  (original: [ TRIAL VERSION
] MHz, overclock: 8%)

    Cachê da CPU:
      Cache L2                                      256 KB  (Asynchronous)

    Propriedades da Placa Mãe:
      ID da Placa Mãe                               <DMI>
      Nome da Placa Mãe                             ASUSTeK COMPUTER INC. X556URK

    Propriedades da BIOS:
      Data da BIOS do sistema                       ü
      Data da BIOS de vídeo                         Desconhecido
```